

HO TECHNICAL UNIVERSITY



FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

**DESIGN AND IMPLEMENTATION OF A SACHET AND BOTTLED WATER
DISTRIBUTION SYSTEM USING MOBILE AND LOCATION-BASED
TECHNOLOGIES**

GRANT AUGUSTA - 0323080015

AXAME BLESS - 0323080500

ADOM TIDY JOAN - 0323080282

TAHIRU ARAFAT - 0323080130

SEPTEMBER, 2026

HO TECHNICAL UNIVERSITY

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

**DESIGN AND IMPLEMENTATION OF A SACHET AND BOTTLED WATER
DISTRIBUTION SYSTEM USING MOBILE AND LOCATION-BASED
TECHNOLOGIES**

GRANT AUGUSTA - 0323080015

AXAME BLESS - 0323080500

ADOM TIDY JOAN - 0323080282

TAHIRU ARAFAT - 0323080130

**PROJECT WORK SUBMITTED TO THE DEPARTMENT OF COMPUTER
SCIENCE, FACULTY OF APPLIED SCIENCES AND TECHNOLOGY, IN
PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
HIGHER NATIONAL DIPLOMA (HND) IN INFORMATION AND
COMMUNICATION TECHNOLOGY**

SEPTEMBER, 2026

DECLARATION

We hereby declare that this submission is our own work towards the award of Higher National Diploma in Information Technology and that, to the best of our knowledge, it contains no material previously published by another person nor material which has been accepted for the award of any other certificate of the University or any institution, except where due acknowledgement has been made in the text.

GRANT AUGUSTA

(0323080015)

.....

.....

Signature

Date

AXAME BLESS

(0323080500)

.....

.....

Signature

Date

ADOM TIDY JOAN

(0323080282)

.....

.....

Signature

Date

TAHIRU ARAFAT

(0323080130)

.....

.....

Signature

Date

CERTIFICATION

Certified by:

DR. VICTOR AGBESI
SUPERVISOR

.....

.....

Signature

Date

Certified by:

PROF. JOHN COKER AYIMAH
HEAD OF DEPARTMENT

.....

.....

Signature

Date

DEDICATION

We dedicate this work first to God Almighty for granting us the wisdom, health, and grace to complete this journey.

We also dedicate it to our supervisor, Head of Department and lecturers for their invaluable academic guidance also to our beloved parents for their endless sacrifices, prayers, and support; and to our colleagues and friends for their camaraderie and encouragement throughout our studies.

ACKNOWLEDGEMENT

We wish to express our profound gratitude to God almighty for making this study a success. A special thanks to our supervisor for his support and professional advice that made this paper a whole. We sincerely appreciate your contribution, suggestion and guidance. We also want to thank the entire staff of Computer Science Department for their support. Our sincere thanks also go to Ho Technical University for giving us the opportunity to study in this prestigious institution as well as providing us with all resources and facilities needed to make our study worthwhile. Finally, we express our heartfelt to our parents, siblings, friends and our various religious leaders for all their support and prayers during the course of study.

ABSTRACT

Access to sachet and bottle water in Ho Municipality depends on informal, single vendor channels such as phone calls and WhatsApp messages, leaving customers unable to compare nearby producers or pay electronically, while small producers lack a shared channel to reach customers beyond their neighborhood. This project designed and implemented a Sachet and Bottled Water Distribution System, a cross platform mobile application allowing multiple registered vendors in Ho Municipality to be discovered, ordered from, and paid by customers through one shared platform. The study introduces a multi-vendor delivery platform to address the gap by designing an iterative, incremental methodology to build the system. The customer and vendor applications were built with Flutter for Android and iOS from a single codebase while the server side used the PHP Laravel framework with a MySQL database verified to satisfy the Second Normal Form. Customers locate nearby vendors through their device's own mobile location services rather than dedicated GPS hardware, add products to a cart, and pay through Paystack while vendors manage products, delivery zones, and orders, and an administrator approves vendors and monitors activity through a Google Maps based dashboard. Testing confirmed that registration, discovery, checkout, payment, vendor approval, and delivery zone enforcement all function as designed, though automated testing remains future work. The system shows that proven technologies can replace fragmented manual ordering with a functioning, multi-vendor alternative.

Keywords: Sachet Water, Bottled Water, Mobile Application, Multi-Vendor Marketplace, Location-Based Services.

TABLE OF CONTENTS

DECLARATION	ii
CERTIFICATION.....	iii
DEDICATION	iv
ACKNOWLEDGEMENT	v
ABSTRACT	vi
TABLE OF CONTENTS	vii
LIST OF TABLES	x
LIST OF FIGURES.....	xi
ABBREVIATIONS AND DEFINITIONS.....	xii
CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Project	1
1.2 Statement of the Problem.....	3
1.3 Aim and Objectives of the Project.....	4
1.4 Significance of the Project	5
1.5 Scope of the Project.....	6
1.6 Organisation of the Project	7
CHAPTER TWO: LITERATURE REVIEW	9
2.1 Review of Related Works	9
2.2 Review of Related Systems.....	11
2.3 Existing System.....	14
CHAPTER THREE: DESIGN AND METHODOLOGY	20
3.1 The Proposed System	20
3.2 Selection of Technologies and Tools.....	24

3.3 Architecture of the System.....	25
3.4 Design of the System.....	26
CHAPTER FOUR: SYSTEM DEVELOPMENT AND TESTING.....	33
4.1 Implementing of the Design.....	33
4.2 Testing of the New System	50
4.3 Documentation	53
4.4 Limitation of the Project	53
CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS	56
5.1 Conclusion	56
5.2 Recommendations	57
REFERENCE.....	59
APPENDIX A: PROJECT PLAN.....	61
A.1: PROJECT PLAN AND TIMELINE.....	61
A.2: TOOLS AND RESOURCES USED	63
APPENDIX B: USER INTERFACE	63
B.1: Splash/ Onboarding Screen.....	64
B.2: Customer Registration Form.....	65
B.3: Vendor Business Profile Submission	67
B.4: Order Screen	69
B.5: Vendor rating and Review Screen.....	70
B.6: Admin User Account Management Screen	71
B.7: Admin Pending Vendor Approvals list	72
APPENDIX C: USER DOCUMENTATION.....	73
C.1: CUSTOMER QUICK-START GUIDE	73

C.2: VENDOR QUICK-START GUIDE	73
C.3: ADMINISTRATOR QUICK-START GUIDE.....	74

LIST OF TABLES

Table 2.1 Summary of Related Multi-Vendor E-Commerce and Delivery Systems	12
Table 4.1 Sample System Test Cases	51
Table A.1 Project Plan and Timeline	61
Table A.2 Tools and Resources Used	63

LIST OF FIGURES

Figure 3.1 Layered System Architecture of the Sachet and Bottled Water Distribution System	26
Figure 3.2 Flowchart of the Order Placement and Delivery Process	27
Figure 3.3 Use Case Diagram of the Sachet and Bottled Water Distribution System.....	28
Figure 3.4 Sequence Diagram for Placing and Paying for an Order	29
Figure 3.5 Class Diagram of the Sachet and Bottled Water Distribution System.....	30
Figure 3.6 Entity-Relationship Diagram (2NF) of the marketplace Database.....	32
Figure 4.1 Vendor Registration and Login	36
Figure 4.2 Vendor Product Code	37
Figure 4.3 Order Placement and Paystack Payment Code.....	38
Figure 4.4 Admin Vendor-Approval Page.....	38
Figure 4.5 Eloquent Model / Database Migration for the Orders Table.....	39
Figure 4.6 Customer App Nearby Vendors / Home Screen.....	41
Figure 4.7 Customer App Product Listing and Cart.....	42
Figure 4.8 Customer App Checkout and Payment Method Selection	43
Figure 4.9 Vendor App Money Withdrawal Screen.....	44
Figure 4.10 Admin Web Dashboard Google Maps Vendor/Order Monitoring	45
Figure 4.11 Vendor Web Dashboard Google Maps Vendor/Order Monitoring	46
Figure 4.12 Homepage of Application	47
Figure 4.13 Homepage of Vendor Account.....	48
Figure 4.14 Admin Paystack Successful Dashboard.....	49

ABBREVIATIONS AND DEFINITIONS

API: Application Programming Interface is a set of rules that allows the mobile app, the backend, and external services such as Paystack and Google Maps to communicate with one another.

ATM: Automated Teller Machine is used here to refer to direct debit/credit card payment processed through a bank card.

ERD: Entity-Relationship Diagram is a diagram showing the tables in the database and how they relate to one another.

GPS: Global Positioning System is a satellite-based positioning system; this project deliberately uses a smartphone's broader mobile location services rather than GPS hardware alone.

JSON: JavaScript Object Notation is a lightweight, text-based data format used to exchange information between the mobile app, the backend, and payment gateways.

KYC: Know Your Customer is the identity/business verification information a vendor submits before being approved on the platform.

MoMo: Mobile Money is mobile-phone-based electronic money transfer service.

MySQL: an open-source relational database management system used to store the system's data.

ORM: Object-Relational Mapping is a technique (implemented via Laravel's Eloquent) that lets the application interact with database tables using programming-language objects.

PHP: Hypertext Preprocessor is the server-side scripting language used to build the system's backend.

REST: Representational State Transfer is an architectural style used to design the Application Programming Interface that connects the mobile app to the backend.

SDK: Software Development Kit is a collection of tools used to build for a specific platform, such as the Google Maps SDK for Flutter.

SQL: Structured Query Language is the language used to define and query the MySQL database.

TLS/HTTPS: Transport Layer Security / Hypertext Transfer Protocol Secure is the encryption protocol used to secure communication between the mobile app and the backend.

2NF: Second Normal Form a level of database normalization in which every non-key attribute depends on the whole of a table's primary key.

CHAPTER ONE

INTRODUCTION

1.1 Background of the Project

Access to safe and affordable drinking water remains one of the most pressing development challenges in Ghana, mainly in urban and peri-urban communities where public pipe-borne water supply is often irregular. Over the past three decades, sachet water, popularly referred to as “pure water,” has emerged as the dominant alternative source of drinking water for many Ghanaian households. Empirical evidence shows that sachet water consumption in Ghana rose sharply between 2010 and 2017, becoming the primary drinking water source for about a third of Ghanaian households by the end of that period, with growth recorded across both low- and high-income groups (Moulds et al., 2022). This growth has been accompanied by a corresponding expansion in the number of registered packaged water production companies across the country, most of which rely on informal, walk-in, or telephone-based ordering systems to reach their customers (Wardrop et al., 2017).

Ho Municipality, the capital of the Volta Region, has witnessed a similar rise in the number of small and medium-scale sachet and bottle water production companies competing to serve a growing population of students, workers, event organisers, and households. In practice, however, ordering sachet and bottled water in bulk or on a recurring basis in Ho still relies heavily on manual processes: customers must know the physical location or telephone number of a producer, call to check availability, negotiate a delivery time, and pay in cash on delivery. There is currently no unified digital platform through which residents of Ho can compare nearby sachet and bottled water producers, place an order on a mobile device, track delivery coordination, and pay securely online.

At the same time, mobile technology adoption in Ghana continues to grow rapidly. Mobile connections in the country now exceed the total population when multiple-SIM ownership is accounted for, and mobile-money and card-based digital transaction platforms have become deeply embedded in everyday commerce (GeoPoll, 2024; Modern Ghana, 2025). Cross-platform mobile application frameworks such as Flutter have also made it considerably easier and cheaper to build a single application that runs on both Android and iOS devices, while still

communicating with a centralised server-side backend built with widely available and low-cost technologies such as PHP and MySQL (Google, 2024). Similarly, African-focused payment gateways such as Paystack now allow developers to accept card, mobile-money, and bank-transfer payments directly within a mobile application through a well-documented Application Programming Interface (API), removing much of the complexity that previously made online payment integration difficult for small businesses in West Africa (Paystack, 2026; Finhive, 2025).

Against this background, this project proposes the design and implementation of a Water Distribution System for Ho Municipality. Rather than being built around a single water-producing company, the proposed system is conceived as a multi-vendor platform that allows several registered sachet and bottled water companies operating within Ho to list their products, receive orders, and coordinate deliveries through a single mobile application. Instead of relying exclusively on satellite-based Global Positioning System (GPS) hardware, the system relies on the location services already built into modern smartphones, which combine network, Wi-Fi, and assisted positioning data to estimate a customer's location, suggest nearby participating vendors, and support delivery coordination. Orders placed on the platform are settled electronically via an integrated Paystack payment channel, while a PHP and MySQL backend manages vendors, products, orders, and transaction records.

The choice of a multi-vendor, or aggregator-style, model is deliberate. Many mobile ordering and delivery applications reviewed in the literature, particularly student-built food ordering systems developed with Flutter, are designed around a single restaurant or business, with the application acting as little more than an online storefront for one company (see, for example, the single-vendor Flutter food-ordering architectures discussed in recent undergraduate and postgraduate project reports). While such systems are relatively straightforward to build, they do little to help customers compare options or to help smaller producers compete with more established ones. By contrast, an open platform that welcomes multiple registered sachet and bottle water companies more closely resembles the aggregator model used by ride-hailing and food-delivery platforms elsewhere, in which the value of the system to any single customer grows with the number of participating vendors, and the value to any single vendor grows with the number of customers reachable through the shared application.

The decision to rely on the location services already present on ordinary smartphones, rather than a dedicated GPS receiver module, is also informed by the realities of the target market. Nearly all economically active residents of Ho Municipality who own a smartphone already carry a device capable of estimating its own position through a combination of cell-tower triangulation, Wi-Fi access-point matching, and, where available, satellite signals, all exposed to mobile applications through a single, unified location Application Programming Interface (API). Building the system around this existing capability keeps the hardware requirement for both customers and vendors to “any reasonably modern smartphone,” rather than requiring specialised tracking devices, which would be impractical for small sachet and bottle water businesses and individual delivery riders to acquire and maintain.

1.2 Statement of the Problem

Even though the sachet and bottle water industry in Ho Municipality is growing, people still get their water in a very manual way. Most customers who want to buy in bulk have to call someone they know, ask around, or rely on word of mouth. There is no easy way for them to find out which companies are nearby or ready to take their order. Because of this, people usually stick to the same vendor they know, even if there is a better or closer option. At the same time, new or small producers find it hard to reach customers outside their own area.

For the producers, most sachet and bottle water companies in Ho do not have any online shop or website to show what they have in stock, take orders, or plan deliveries. When they get orders by phone, they usually write them down on paper or in a notebook. This can lead to mistakes, late deliveries, or arguments about how much was ordered or paid for. Since people mostly pay in cash when the water is delivered, both the sellers and buyers face risks like theft, getting the wrong change, or not having any proof of payment if there is a problem later.

In other places, there are apps for ordering things like food and groceries, but these usually only let you buy from one shop or a small group of shops. This means customers do not have many choices, and water companies cannot easily reach more people through these apps. There is a need for a mobile app that lets people in Ho find different sachet and bottle water companies near them, place and track their orders, and pay safely online. This project aims to create such a system for Ho Municipality.

Another problem is that after someone orders by phone, it is hard to organize the delivery. There is no system to keep track of where customers live, so delivery people have to rely on directions given over the phone or just knowing the area well. This often causes late deliveries, missed appointments, or wasted trips, especially for new customers or places that are hard to find. Without a digital record of orders, producers also cannot easily see which products are popular, plan how much to make, or know who their best customers are. This makes it hard for small water businesses in Ho to grow and become more organized.

To sum up, people and businesses in Ho do not have one reliable mobile app where they can find different registered sachet and bottle water companies nearby, make and track orders, and pay safely online. At the same time, water companies do not have an easy and affordable way to get digital orders, organize deliveries using customers' phone locations, or handle payments electronically.

1.3 Aim and Objectives of the Project

The aim of this project is to design and implement a sachet and bottle water Distribution System that enhances water ordering, delivery coordination and payment processing for multiple companies and their customers in Ho Municipality, Volta Region of Ghana.

To achieve the stated aim, the following specific objectives were formulated:

1. To design a cross platform that integrate multiple water companies, enabling customers to purchase at their convenience.
2. To implement mobile network-based location services also known as cell-tower triangulation and Wi-Fi positioning within the application to determine customer locations without relying solely on GPS, enabling accurate and efficient delivery coordination.
3. To develop an integrated mobile payment module within the system that allows customers to make secure digital payments.
4. To implement an order management and delivery coordination module that enables water companies to receive, process, assign, and track deliveries in real time.

1.4 Significance of the Project

This project is significant to several stakeholders within Ho Municipality and beyond. For customers, the proposed system removes the uncertainty associated with manual ordering by allowing them to view multiple registered sachet and bottled water companies in one place, compare their offerings, and place an order on a smartphone without needing to know a specific producer's telephone number in advance. The integration of the Paystack payment gateway further improves convenience and safety by allowing customers to pay electronically rather than handling cash at the point of delivery.

For water production companies, particularly small and medium-scale producers that lack the resources to build individual mobile applications, the platform offers an affordable route to digital visibility. By onboarding onto a shared, multi-vendor system rather than being restricted to a single-company application, participating companies can reach customers beyond their traditional radius, receive orders in a structured and traceable format, and reconcile payments more reliably through electronic records generated by the payment gateway.

The project also carries academic and developmental significance. It contributes a practical, Ghana-specific case study on the use of mobile location services, distinct from dedicated GPS-based navigation systems, for coordinating last-mile delivery of a fast-moving consumer good, adding to the growing body of research on mobile commerce and financial technology adoption in Sub-Saharan Africa (Modern Ghana, 2025). The system architecture and design decisions documented in this project may serve as a reference for future student projects or small businesses seeking to build similar multi-vendor delivery platforms for other product categories or other municipalities in Ghana.

Finally, by making it easier for customers to identify and support registered sachet and bottle water producers, the platform indirectly encourages patronage of formally registered companies over unregulated or unlicensed sources, which may contribute in a small way towards improved consumer awareness of water quality and safety within the municipality (Ahiabor & Donkor, 2025).

The project is equally significant from a technical and training perspective. Its implementation requires the practical combination of several skills central to modern software development, including mobile application development with Flutter and Dart; server-side scripting and

Application Programming Interface (API) design with PHP; relational database design and management with MySQL; and third-party payment gateway integration with Paystack. Undertaking the project, therefore, provides direct, hands-on experience in building a complete, end-to-end software system, from requirements gathering and database design through to mobile user interface development, payment processing, and testing, which is directly relevant to careers in software engineering and mobile application development within Ghana's growing technology sector.

1.5 Scope of the Project

The scope of this project is deliberately bounded to keep the work achievable within a final-year timeframe while still producing a functional, demonstrable system. Geographically, it focuses on Ho Municipality in the Volta Region of Ghana, with the pilot vendors, customer accounts, and delivery scenarios used for design and testing drawn from this municipality. Functionally, the system is a mobile application built with Flutter for customers and vendors, backed by a PHP (Laravel) Application Programming Interface and a MySQL database, and it covers customer and vendor registration, product and price listing, mobile-location-based vendor discovery, order placement and status tracking, secure payment through Paystack, and a simple administrative interface for overseeing vendors and orders.

The project does not automate the internal production process of any participating company or attempt a nationwide rollout; it is scoped as a pilot centred on Ho Municipality that could, in principle, be extended to other municipalities in future work. It relies on the location services already embedded in ordinary smartphones rather than dedicated GPS hardware, and the administrative scope is limited to a lightweight dashboard for approving vendors and monitoring orders rather than a full enterprise resource planning or inventory system. Reporting is limited to basic order and sales summaries, and while the system follows reasonable security practices such as password hashing and input validation, a full-scale penetration test or formal security audit falls outside the scope of this academic project.

1.6 Limitation of the Project

A number of factors limit the depth and coverage of this project:

1. Time constraint: the project is undertaken within a single academic year alongside other coursework, which restricts the scale of testing and the number of real vendors and customers that can be involved during development.
2. Device and connectivity dependence: the system assumes that users have access to an Android or iOS smartphone with an active internet connection; residents who rely solely on basic feature phones or who have limited or unstable internet access may not be able to fully benefit from the platform.
3. Location accuracy: because the system relies on a smartphone's built-in location services rather than dedicated GPS receiver hardware, the estimated location of a customer or vendor may occasionally be less precise than a professional-grade navigation system, particularly in areas with weak network coverage.
4. Vendor participation: the usefulness of the multi-vendor model depends on the willingness of sachet and bottle water companies in Ho Municipality to register and actively maintain their listings; the project cannot compel adoption by existing producers.
5. Payment gateway constraints: development and testing of the Paystack integration are largely carried out in a sandbox or test environment; full production deployment would require each participating vendor to complete Paystack's business verification and settlement account setup.
6. Resource and infrastructure limitations: as a student project, the system is tested on a limited number of devices and a small simulated user base rather than under full commercial load, so performance under large-scale, real-world traffic is not exhaustively evaluated.
7. Logistics realism: actual delivery riders are not commercially engaged during the project; delivery coordination features are demonstrated and tested using simulated or volunteer scenarios rather than a live fleet of riders.

Taken together, these limitations mean that the system produced at the end of this project should be understood as a working prototype and proof of concept for a Sachet and Bottle Water Distribution System, rather than as a finished, commercially deployed product. Nevertheless, the prototype is designed to demonstrate, in a practical and verifiable way, that the core objectives of multi-vendor onboarding, mobile-location-based vendor discovery, order coordination, and secure Paystack-based payment can be achieved using the chosen technology stack of Flutter, PHP, and MySQL within the context of Ho Municipality.

1.7 Organisation of the Project

This project report is organised into five chapters.

Chapter One introduces the project by presenting the background to the study, the statement of the problem, the aim and specific objectives, the significance of the project, its scope and limitations, and an outline of how the report is organised.

Chapter Two presents a review of related literature, covering the sachet and bottled water and packaged water industry in Ghana, mobile commerce and location-based service concepts, existing multi-vendor and single-vendor delivery platforms, and payment gateway technologies such as Paystack, in order to situate the project within existing knowledge and to justify the design choices adopted.

Chapter Three describes the methodology adopted for the project, including the system development approach, requirements gathering and analysis, and the design of the system architecture, database, and user interfaces, covering the Flutter mobile application, the PHP-based backend API, the MySQL database schema, the mobile location-based service integration, and the Paystack payment workflow.

Chapter Four presents the system's implementation, including the tools and technologies used, key modules developed, and the results of system testing, along with screenshots and a discussion of how the implemented system satisfies the objectives stated in this chapter.

Chapter Five concludes the report with a summary of the entire project, the conclusions drawn from the design, implementation, and testing processes, and recommendations for future improvements or extensions of the Sachet and Bottle Water Distribution System.

CHAPTER TWO

LITERATURE REVIEW

2.1 Review of Related Works

This section reviews academic and technical literature relevant to the design and implementation of a mobile, location-aware, multi-vendor distribution system, drawing on studies of packaged water in Ghana, mobile application development with Flutter, server-side integration with PHP and MySQL, and mobile commerce more broadly. The intention is not simply to list prior work, but to identify what has already been established, what technology choices have proven effective in comparable systems, and what gap remains for a project focused specifically on multi-vendor sachet and bottle water ordering and delivery in Ho Municipality.

Studies of the packaged water sector in Ghana confirm both the scale and the informality of the industry that this project seeks to digitize. Moulds et al. (2022) traced a sharp national rise in sachet water consumption between 2010 and 2017, showing that reliance on sachet water is no longer confined to low-income households but now cuts across income groups, which supports the assumption in this project that a mobile ordering platform for sachet and bottle water would serve a broad cross-section of residents rather than a narrow market segment. Wardrop et al. (2017) similarly mapped the geographic spread of registered packaged water producers across Ghana and found that most such businesses operate through informal, localised distribution rather than centralized digital channels, a finding that is consistent with the manual, phone-based ordering practices observed in Ho Municipality and described in Chapter One. Ahiabor and Donkor (2025), reviewing microbial safety across sachet water studies in Ghana, further underline the importance of sourcing water from properly registered and monitored producers, reinforcing the value of a platform that only onboards verifiable, registered sachet and bottle water companies rather than anonymous informal sellers.

On the technical side, several recent studies validate the specific technology stack proposed for this project. Nursobah et al. (2024) implemented the front end of an e-commerce fashion application using Flutter and a User-Centered Design approach, and reported a System

Usability Scale score in the “good” category, providing evidence that Flutter is capable of producing a responsive, well-received shopping interface even for developers working on a student or small-team budget. Related work reviewed by IRJMETs (2024) on a Flutter-based food-ordering system demonstrated a complete order life cycle from browsing and cart management through to order fulfilment and delivery-personnel assignment built on a similar cross-platform mobile approach to the one proposed in this project, confirming that Flutter is a practical choice for building the customer-facing ordering flow of a delivery-oriented application.

Literature on location-based mobile services also supports the specific design decision, explained in Chapter One, to use a smartphone’s built-in location services rather than a dedicated GPS receiver. Studies combining Flutter front ends with PHP-based backends and Google’s Maps and location Application Programming Interfaces have shown that consumer-grade smartphone location data is sufficiently accurate for tasks such as identifying nearby vendors, estimating travel distance, and supporting delivery coordination, even though it is less precise than dedicated survey-grade GPS equipment. This body of work indicates that network- and device-based location services, while occasionally imprecise in areas of weak signal coverage, are an appropriate and widely used foundation for consumer delivery applications rather than a compromise unique to this project.

Finally, broader research and industry analysis on mobile money and digital payment adoption in Ghana (GeoPoll, 2024) demonstrates that the enabling conditions for an in-app payment feature are already largely in place: mobile connections in Ghana now exceed the total population when multiple-SIM ownership is considered, and digital payment channels, including card and mobile-money rails of the kind offered through Paystack, have become an accepted part of everyday commerce. Taken together, the reviewed literature confirms that (a) the sachet and bottle water sub-sector in Ghana is large, growing, and still served mainly through informal, single-vendor, manual channels; (b) Flutter, PHP, and MySQL form a proven and adequately documented technology combination for building a mobile ordering and delivery system; and (c) smartphone-based location services and mobile payment gateways are mature enough to support the kind of multi-vendor, location-aware system proposed in this project. The remaining gap, which this project addresses, is the absence of a system that brings

these proven components together specifically for multiple sachet and bottle water companies operating within a single Ghanaian municipality.

2.2 Review of Related Systems

Beyond the academic literature, it is useful to examine real-life multi-vendor e-commerce and on-demand delivery platforms that share structural similarities with the proposed Sachet and Bottle Water Distribution System, since these systems have already been tested at scale with real customers, vendors, and delivery logistics. Ten such systems, ranging from global horizontal marketplaces to Ghana- and Africa-focused delivery aggregators, are reviewed below.

Table 2.1 below summarizes the ten related systems reviewed in this section, showing the method used by each system, its key strengths, and its notable limitations, to support direct comparison with the design of the proposed Sachet and Bottle Water Distribution System.

Table 2.1: Summary of Related Multi-Vendor E-Commerce and Delivery Systems

System	Method Used	Strengths	Limitations
Amazon	Commission-based, horizontal B2C/B2B marketplace where third-party sellers list products alongside Amazon's own inventory; uses shared logistics (Fulfilment by Amazon) and centralised checkout.	Very wide product selection; strong logistics and delivery network; high buyer trust and buyer-protection policies.	High seller commissions and fulfilment fees; intense competition among sellers; not tailored to any single product category such as water.

Alibaba	B2B wholesale marketplace connecting manufacturers and bulk buyers, with request-for-quotation and bulk-pricing tools.	Enables large-scale sourcing and bulk ordering; strong supplier verification tools.	Designed for bulk/wholesale trade rather than last-mile consumer delivery; minimum order quantities can be too large for individual consumers.
Etsy	Niche, vertical C2C/B2C marketplace focused on handmade and vintage goods, with individual seller storefronts under one platform.	Strong sense of community and trust within a specific niche; simple seller onboarding.	Narrow product category focus; limited real-time location-based delivery coordination; delivery timelines can be long.
Jumia	Horizontal e-commerce marketplace with its own logistics arm and JumiaPay for order settlement, allowing many independent vendors to sell through one app.	Wide reach across Ghana and other African countries; multiple payment options including mobile money and pay-on-delivery.	General marketplace not specialised for fast-moving daily essentials such as sachet and bottle water; delivery times can be slower than on-demand services.
Jumia Food (discontinued)	Restaurant- and vendor-aggregator model using an in-house delivery fleet to fulfil food orders	Simple, familiar ordering flow; benefited from Jumia's existing brand and payment infrastructure.	Shut down across seven African markets in 2023 due to weak unit economics, showing the financial fragility

	placed through the app.		of the pure aggregator-delivery model (Techpoint Africa, 2026).
Jiji Ghana (formerly competing with Tonaton)	Free classified-listings (C2C) marketplace where sellers post items and buyers contact them directly to arrange payment and delivery.	Very low barrier to entry for small and informal sellers; large existing user base in Ghana (Statista, 2025).	No built-in order tracking, in-app payment, or location-based vendor discovery; transactions and delivery are arranged informally outside the app.
Bolt Food	On-demand food-delivery aggregator that reuses Bolt's existing ride-hailing driver network and in-app GPS tracking for order assignment.	Fast delivery through an existing driver network; live order tracking; has operated profitably in Ghana for several years (TechCabal, 2025).	Limited to the food category; withdrew entirely from the Nigerian market in December 2023 after struggling with profitability (TechCabal, 2025).
Glovo	Multi-category, on-demand delivery aggregator covering food, groceries, and courier requests, matched to riders in real time.	Broad category coverage beyond food; wide brand presence across several countries.	Exited the Ghanaian market in 2024, illustrating how difficult it can be to sustain a multi-category delivery aggregator in this region (TechCabal, 2025).

Chowdeck	Food- and retail-delivery aggregator that uses geotagging to match every order to the nearest available rider and forecasts demand to allocate riders ahead of peak periods.	Notably fast delivery and rider-matching efficiency; has remained profitable in each city it enters, including a fast, well-received expansion into Accra in 2025 (Techmoonshot, 2026; Afridigest, 2025).	Growth has relied partly on exclusive restaurant partnerships that can lock competitors out of popular vendors, reducing customer choice (TechCabal, 2025).
mPharma	Vendor-managed inventory platform that links pharmacies, hospitals, and drug suppliers, coordinating stock levels and distribution across a multi-party supply network rather than a single seller.	Improves medicine availability and pricing consistency across a network of independent pharmacies; has scaled to multiple African countries since 2013 (TechCrunch, 2021).	Focused on pharmaceutical supply-chain coordination between businesses rather than direct, on-demand consumer ordering and delivery.

2.3 Existing System

While Section 2.2 reviewed large, well-known multi-vendor platforms operating across various sectors, this section turns specifically to the existing system currently used to order and distribute sachet and bottle water in Ho Municipality and in Ghana more broadly, since it is this manual, largely offline system that the proposed application is intended to replace. In the absence of a shared digital platform dedicated to sachet and bottle water, residents and businesses across Ghana rely on a patchwork of informal, single-vendor channels operated directly by individual producers or informal resellers.

Several real-life examples illustrate how this existing system typically works in practice. Ghana Water Limited, the state-owned utility responsible for treating and distributing pipe-borne water to about 1.01 million metered customers nationwide, represents the formal, large-scale end of water distribution in the country, but its services are limited to fixed pipe-borne connections and do not extend to on-demand ordering of sachet or bottled water (Wikipedia, 2025). At the informal end of the spectrum, small water-delivery businesses such as the now-defunct Wewater Ghana and its contemporary Wasser Water operated (and, in Wasser Water's case, continue to operate) through Instagram advertising and direct WhatsApp messaging: a customer sees an advertisement, sends a WhatsApp message stating the brand and quantity required, receives a price quotation and delivery window by chat, pays through a manually shared mobile-money number, and shares their delivery location as a Google Maps link before waiting for a phone call announcing the rider's arrival (Quartey, 2023). Similarly, bulk and tanker water-delivery services such as those offered by GhanaResidential.com and Water Supply Tanker Accra operate primarily through phone calls and WhatsApp messages, with pricing that depends on tanker size and delivery distance being negotiated case by case rather than displayed transparently in advance (GhanaResidential.com, 2025). Individual sachet and bottle water producers, such as Voltic Cool Pac, similarly rely on direct calls and social media pages such as Facebook to advertise production, wholesaling, and delivery services within their operating area, rather than a searchable, location-aware ordering platform (Voltic Ghana Ltd, 2025).

This pattern informal advertising, manual phone or chat-based ordering, negotiated rather than fixed pricing, and cash or manually confirmed mobile-money payment is consistent with what

was observed in Ho Municipality and described in the Statement of the Problem in Chapter One, and it mirrors the broader finding in the literature that most registered packaged water producers in Ghana rely on informal, localized distribution channels rather than centralized digital platforms (Wardrop et al., 2017). One water-delivery operator serving Accra even acknowledges publicly that, before its own service existed, customers had “to wait for water that may or may not come and to pay whatever price the trucker decides when they arrive,” a candid illustration of the pricing opacity and unreliability that characterize the existing system (Clearwater Africa, 2026).

2.3.1 Component of Existing System

Based on the patterns observed above, the existing system for ordering and distributing sachet and bottle water in Ho Municipality, and in Ghana generally, can be broken down into the following components:

1. Individual sachet and bottle water producers or resellers, each operating their own independent, single-vendor sales channel (a phone line, a WhatsApp number, or a social media page such as Facebook or Instagram).
2. Customers, who must already know or discover a specific producer’s contact details through word of mouth, a signboard, or an advertisement, since there is no shared directory of nearby vendors.
3. An informal communication channel typically a voice call or a WhatsApp chat through which the customer states the product and quantity required and receives a price and an estimated delivery time.
4. Delivery personnel, usually riders, drivers, or tricycle operators engaged directly by the producer, who are given a verbal address or a shared map pin and must locate the customer without route optimization or centralized tracking.
5. A manual payment step, typically cash on delivery or a mobile-money transfer to a personal or business mobile-money number, confirmed verbally or by screenshot rather than through an integrated payment gateway.

6. Paper notebooks or, at best, simple spreadsheets kept by individual producers to record orders, with no shared database, order history, or analytics across the sector.

2.3.2 Process of the System

The typical order-to-delivery process under the existing system follows a broadly consistent, largely manual sequence:

1. Discovery: the customer becomes aware of a specific sachet and bottle water producer through word of mouth, a roadside signboard, or a social media advertisement, since no platform lists multiple nearby producers together.
2. Contact: the customer calls or sends a WhatsApp message to the producer's personal or business phone number, stating the product, quantity, and delivery address.
3. Availability check and price quotation: a staff member or the producer personally checks whether enough stock is available and quotes a price, which may vary by customer, distance, or time of day.
4. Delivery-window negotiation: the two parties agree, by voice or chat, on an approximate delivery time, without any automated scheduling or route planning.
5. Record-keeping: the transaction, if recorded at all, is noted in a notebook or informal ledger kept by the individual producer, with no shared record accessible to the customer or to any other vendor.

2.3.3 Problems of the Existing System

Building on the problem statement in Chapter One, and on the concrete examples reviewed above, the existing manual system for ordering and distributing sachet and bottle water in Ho Municipality exhibits the following specific problems:

1. Vendor lock-in and lack of comparison: because each producer operates its own isolated contact channel, customers cannot easily discover or compare multiple nearby vendors, and tend to repeatedly patronise a single familiar producer even when a closer or more available alternative exists.
2. Opaque and inconsistent pricing: as illustrated by customers of tanker water services who report paying “whatever price the trucker decides when they arrive” (Clearwater Africa, 2026), prices are frequently negotiated case by case rather than displayed transparently in advance.
3. Unreliable and imprecise delivery coordination: verbal addresses and shared map pins, in the absence of a proper in-app location and route-matching system, often require follow-up phone calls and can lead to delayed or missed deliveries, particularly for first-time customers.
4. Cash-handling and payment-verification risk: reliance on cash on delivery or manually confirmed mobile-money transfers exposes both customers and producers to the risk of short-changing, theft, or disputed payments, with no automated, verifiable transaction record.
5. Limited business intelligence for producers: because orders are recorded, if at all, in personal notebooks rather than a shared digital system, individual sachet and bottle water companies struggle to analyse demand patterns, plan production, or identify their most valuable customers.
6. No mechanism for multi-vendor competition or growth: unlike the aggregator platforms reviewed in Section 2.2, the existing system offers no shared channel through which several registered sachet and bottle water companies in Ho Municipality could reach a common pool of customers, meaning that smaller or newer producers remain limited to whatever customer base they can build individually.

These problems, evident both in the direct observation of practice in Ho Municipality and in comparable examples drawn from elsewhere in Ghana, provide the concrete justification for the Sachet and Bottle Water Distribution System proposed in this project: a single, multi-vendor mobile platform that replaces informal, single-channel ordering with structured vendor discovery based on mobile-device location, transparent pricing, trackable order status, and secure, verifiable Paystack-based payment.

CHAPTER THREE

DESIGN AND METHODOLOGY

3.1 The Proposed System

The proposed Sachet and Bottled Water Distribution System is a cross-platform mobile solution that brings together three categories of users which are the Customer, the Vendor, and the System Administrator on one shared platform, allowing several registered sachet and bottle water companies operating in Ho Municipality to be discovered, ordered from, and paid by customers through a single application rather than each company running its own isolated ordering channel. The customer- and vendor-facing interfaces are built once with Flutter and Dart and compiled to run natively on both Android and iOS, so that a single codebase serves the two dominant smartphone platforms rather than two separate native applications (Nursobah et al., 2024). Because a meaningful share of residents in Ho Municipality, particularly older customers and small-scale institutional buyers, In this sense, “cross-platform” in this project is used in the broader sense of reaching customers across multiple types of devices Android and iOS operating systems alone.

The Customer uses the system to locate nearby vendors and order either sachet water (pure water sold in small plastic sachets) or bottle water (water sold in sealed plastic bottles of various sizes), through the location services already built into their smartphone combining network, Wi-Fi, and satellite signals through a single location Application Programming Interface rather than a dedicated GPS receiver. The Vendor, representing a registered sachet or bottle water production company, uses the system to list products, manage incoming orders, and update delivery status once an order has been dispatched to the customer. The System Administrator oversees the platform as a whole through a web-based dashboard, embedding the Google Maps Platform, on which vendor and customer locations can be visually pinpointed, vendor accounts can be approved or suspended, and overall order activity across Ho Municipality can be monitored. Once an order is placed, payment is settled electronically through one of three channels: the Paystack payment gateway (which itself supports card and

MTN Mobile Money payments), giving customers flexibility to pay however is most convenient for them (Faciotech Blog, 2026).

3.1 System Specification

This section specifies, in concrete terms, what the system must be able to do (functional requirements) and the quality attributes it must exhibit while doing so (non-functional requirements). These requirements were derived from the problems identified with the existing manual system in Chapter Two, from the features observed across the related systems reviewed in Section 2.2, and from the three user roles such as the Customer, Vendor, and Administrator are the once the system is built around.

3.1.1 Functional Requirements

The functional requirements outline the specific actions the system must be able to perform to fulfil its purpose. These include:

1. Account Registration and Login: the system must allow a Customer, Vendor, or Administrator to register an account and log in securely using a phone number or email address and password appropriate to their role.
2. Location-Based Vendor Discovery: the system must capture a customer's device location through mobile location services and display a list or map of nearby registered sachet and bottle water vendors within Ho Municipality.
3. Product Catalogue Management: the system must allow a Vendor to create and manage a catalogue of sachet water and bottle water products, each with a name, price, stock quantity, and availability flag.
4. Cart Management: the system must allow a customer to add one or more sachet or bottle water products from a chosen vendor to a personal cart and adjust quantities before checkout.
5. Order Placement: the system must allow a customer to convert a cart into an order, review the order summary (subtotal, delivery fee, and total), and confirm a delivery location.

- 6. Vendor Approval Workflow:** the system must require a new Vendor account to submit a business profile for review, and must allow an Administrator to approve or reject the application with a review note before the vendor can list products.
- 7. Delivery Zone Definition:** the system must allow a Vendor to define one or more delivery zones, each with a centre point and a coverage radius, so that the system can determine whether a customer's location falls within a deliverable area.
- 8. Multi-Channel Payment Processing:** the system must allow a customer to pay for an order using Paystack (card or mobile money) transaction, and must return an electronic confirmation of successful payment.
- 9. Order Tracking:** the system must allow a customer to track the status of a placed order (received, confirmed, out for delivery, delivered) in real time.
- 10. Order and Delivery Management:** the system must allow a Vendor to view, accept, and update the status of incoming orders (received, confirmed, out for delivery, delivered) through to completion.
- 11. Vendor Rating:** the system must allow a customer to rate and briefly review a Vendor after an order has been delivered.
- 12. Administrative Oversight:** the system must provide an Administrator dashboard, embedding the Google Maps Platform, that pinpoints the locations of customers and vendors, approves or suspends vendor accounts, and displays basic order and sales analytics.
- 13. Notifications:** the system must send notifications (in-app or SMS) to customers and vendors at key stages of the order lifecycle.

3.1.2 Non-Functional Requirements

Non-functional requirements specify the qualities the system must have to function effectively and efficiently. These include:

1. **Compatibility:** the mobile application must run on Android and iOS smartphones through a single Flutter codebase.
2. **Usability:** the interface must be simple and intuitive enough for a first-time Customer or Vendor to complete their main task placing an order, or listing a product in a few steps, following a User-Centered Design approach consistent with related Flutter e-commerce work (Nursobah et al., 2024).
3. **Performance:** common actions such as loading nearby vendors or submitting an order must complete within a few seconds under normal mobile network conditions in Ho Municipality.
4. **Security:** passwords must be stored as salted hashes, all traffic between the mobile application and the PHP backend must be encrypted using HTTPS/TLS, and payment details must never be stored directly on the platform's own servers, relying instead on tokenized transactions through Paystack.
5. **Reliability and Availability:** the system must remain available for ordering during normal business hours with minimal downtime, and must handle failed payment attempts gracefully without losing order data.
6. **Data Privacy and Legal Compliance:** collection and processing of customers' personal data and location information must comply with Ghana's Data Protection Act, 2012 (Act 843), including obtaining consent before accessing device location (Wikipedia, 2025b).
7. **Scalability:** the underlying PHP and MySQL backend must be structured so that additional sachet and bottle water vendors and customers in Ho Municipality, and potentially other municipalities in future, can be onboarded without redesigning the core system.

8. Maintainability: the codebase must be modular and documented so that future student developers or maintainers can extend individual components, such as adding a new payment channel, without reworking the entire system.

3.2 Selection of Technologies and Tools

The technologies chosen for the development of this system were selected for being well documented, actively maintained, affordable for a student project, and directly suited to the functional and non-functional requirements specified in Section 3.1. The primary technologies include:

1. Frontend: Flutter and Dart, used to build a single, cross-platform codebase for the Customer and Vendor mobile applications that compiles natively to both Android and iOS, shown in related literature to produce responsive, well-received e-commerce interfaces (Nursobah et al., 2024; IRJMETS, 2024).
2. Backend: PHP, built on the Laravel framework, used to construct the server-side REST Application Programming Interface that handles authentication, vendor and product management, order processing, and communication with external payment and mapping services. Laravel's Eloquent ORM and migration system are used to define and evolve the database schema, and Laravel Sanctum issues the personal access tokens that authenticate mobile app requests (PHP Group, 2023; Laravel, 2026).
3. Database: MySQL, used as the relational database management system storing users, vendors, products, orders, and payment records, chosen for its wide support, low cost, and compatibility with PHP (Oracle Corporation, 2023).
4. Payment Gateways: The Paystack Application Programming Interface, supporting card and MTN Mobile Money payments supporting together with support for ATM/debit and credit card payment through Paystack's card-processing rails (Paystack, 2026; Faciotech Blog, 2026).

5. Location and Mapping: The Google Maps Platform (Maps SDK, Geocoding, and Places Application Programming Interfaces), used on the Administrator's web dashboard to pinpoint and visualise the locations of customers and vendors, and within the mobile app to translate device coordinates into readable addresses and estimate distance between a customer and nearby vendors.
6. Development Environment: XAMPP, used as the local development environment bundling PHP, MySQL (via MariaDB), and Apache for building and testing the backend before deployment.
7. API Testing: Postman, used to design, document, and test the REST API endpoints independently of the mobile front end during development.
8. Integrated Development Environments: Visual Studio Code and Android Studio, used for writing Dart/Flutter code and for managing Android emulators and iOS simulators respectively.
9. Version Control: Git and GitHub, used to keep a documented history of changes to the codebase throughout development.
10. Prototyping: Figma, used to design and prototype the mobile application's user interface and user experience before implementation began.

3.3 Architecture of the System

The proposed system follows a layered client-server architecture, which separates the user-facing applications from the business logic and from data storage, allowing each layer to be developed, tested, and scaled independently. At the presentation layer sit the Flutter-based Customer and Vendor mobile applications (running on Android and iOS) and the Administrator's web-based dashboard embedding the Google Maps Platform. All of these clients communicate over HTTPS with a single application layer, consisting of a PHP REST API that implements authentication, vendor and product management, order processing, location-based vendor matching, and coordination with external services. The application layer, in turn, reads from and writes to a MySQL database at the data layer, and communicates

with three external services: the Paystack for settling orders, and the Google Maps Platform for geocoding and map visualisation. Figure 3.1 illustrates this architecture.

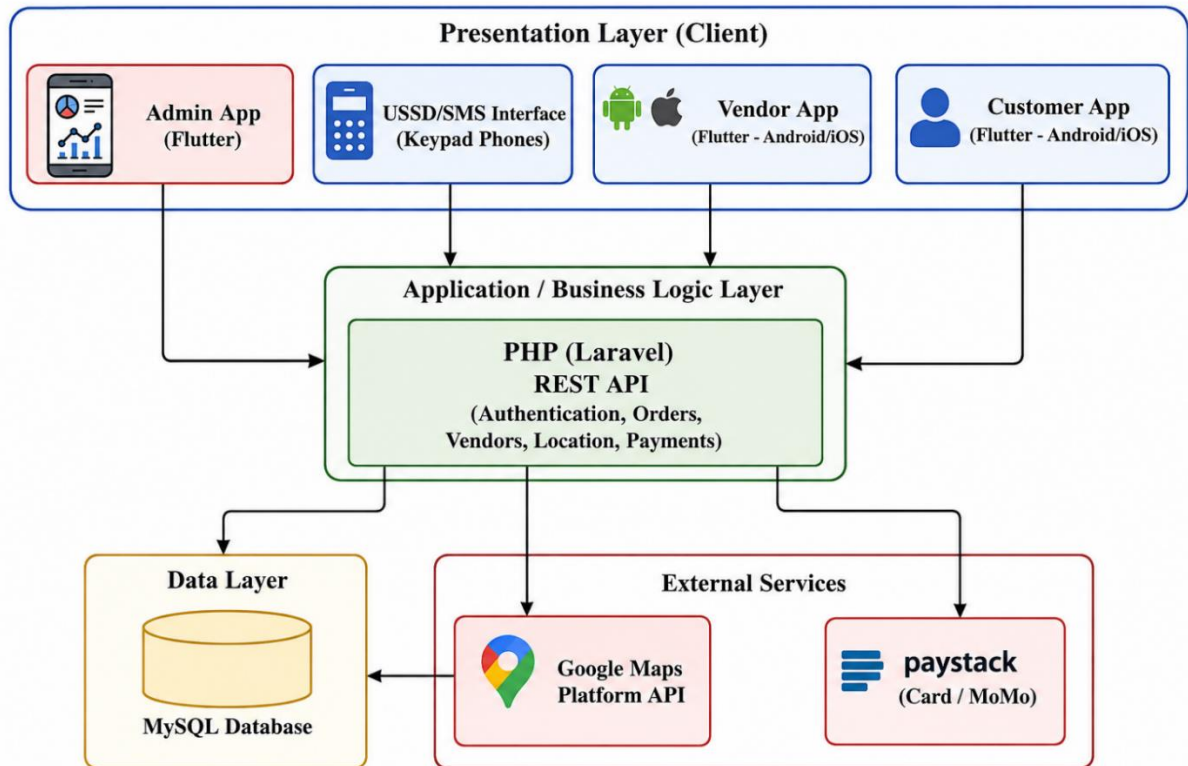


Figure 3.1: Layered System Architecture of the Sachet and Bottled Water Distribution System

3.4 Design of the System

3.4.1 Flow Chart, Use Case, Sequence, and Class Diagrams

This subsection presents four complementary diagrams that together describe the behaviour and structure of the proposed system: a flowchart of the overall ordering process, a use case diagram showing how each of the three actors who are Customer, Vendor, and Administrator interacts with the system, a sequence diagram detailing the order-placement and payment process, and a class diagram showing the main object types and their relationships. Figure 3.2 presents the flowchart of the ordering process from the Customer's point of view, from opening the application to confirming receipt of a delivered order.

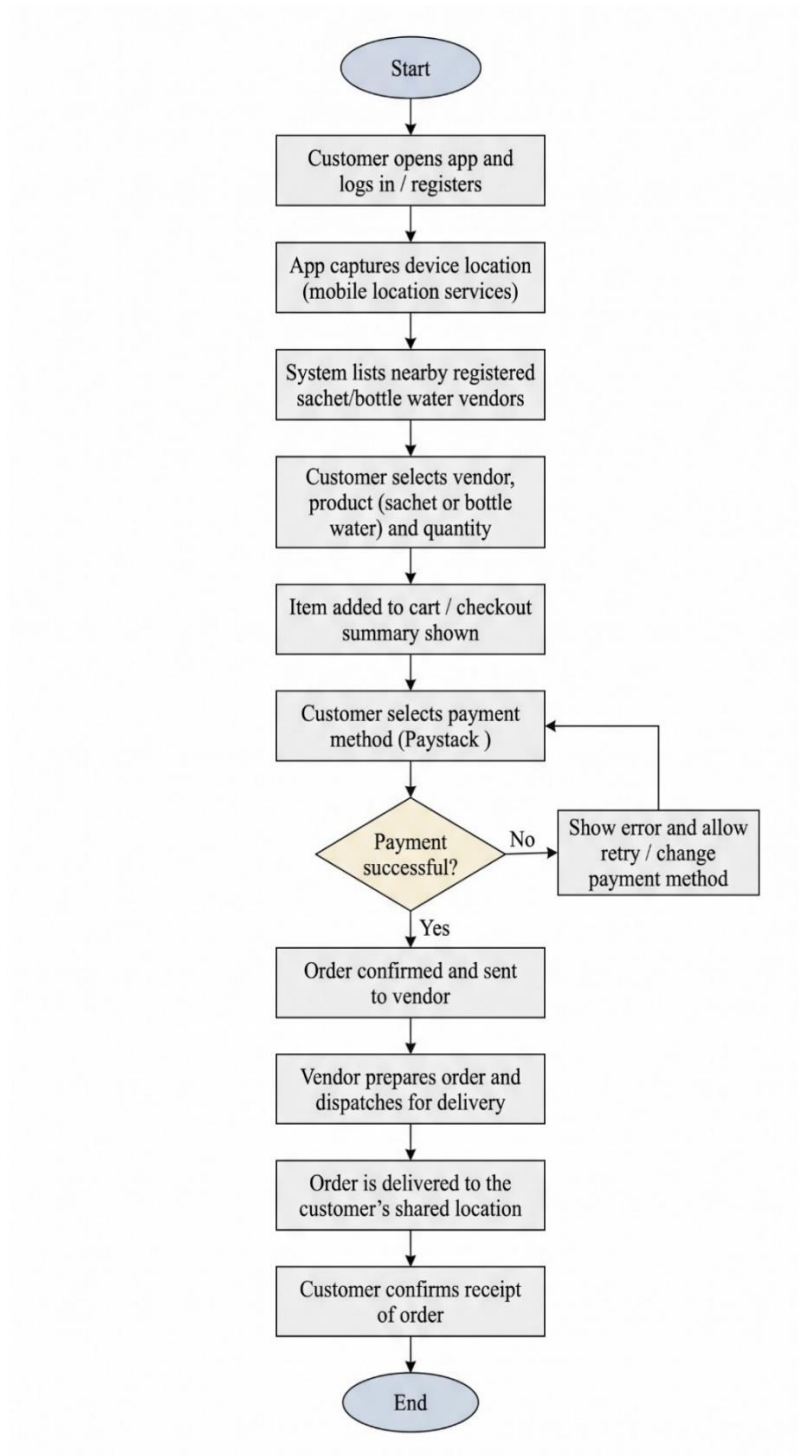


Figure 3.2: Flowchart of the Order Placement and Delivery Process

Figure 3.3 presents the use case diagram, showing the three system actors and their principal use cases.

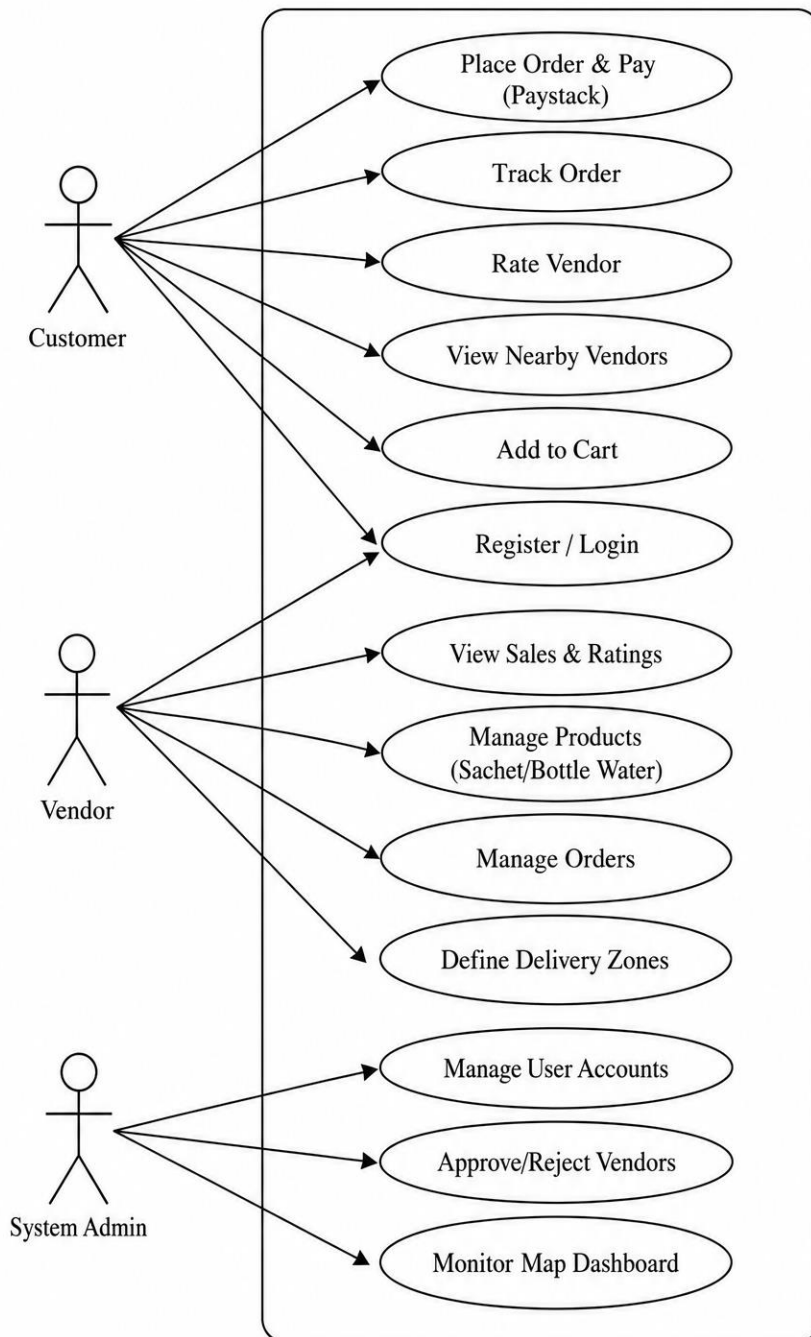


Figure 3.3: Use Case Diagram of the Sachet and Bottled Water Distribution System

Figure 3.4 shows how the Customer, mobile app, API, database, Paystack, and Vendor interact during order placement and payment

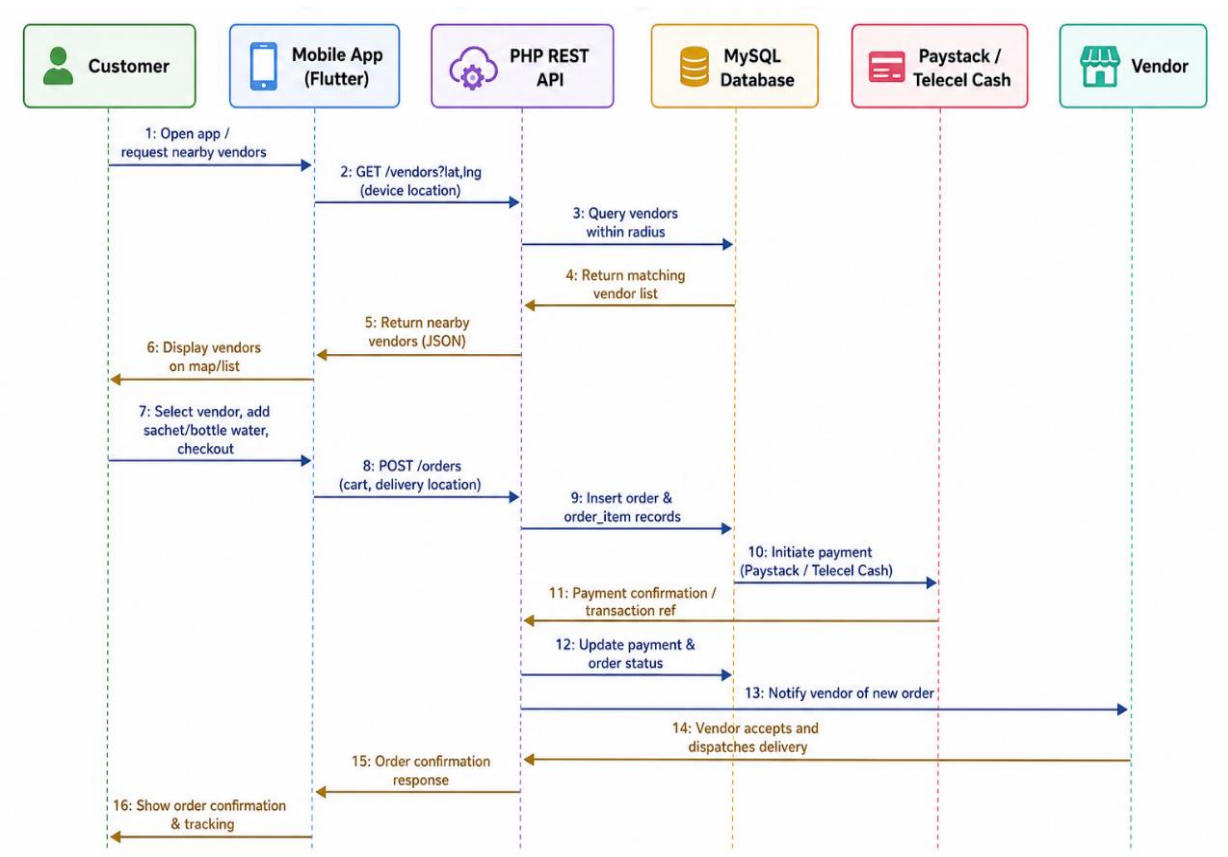


Figure 3.4: Sequence Diagram for Placing and Paying for an Order

Figure 3.5 presents the class diagram, showing the main classes in the system for User (with Customer, Vendor, and Admin specialisations), Product, Order, OrderItem, and Payment together with their key attributes, operations, and relationships.

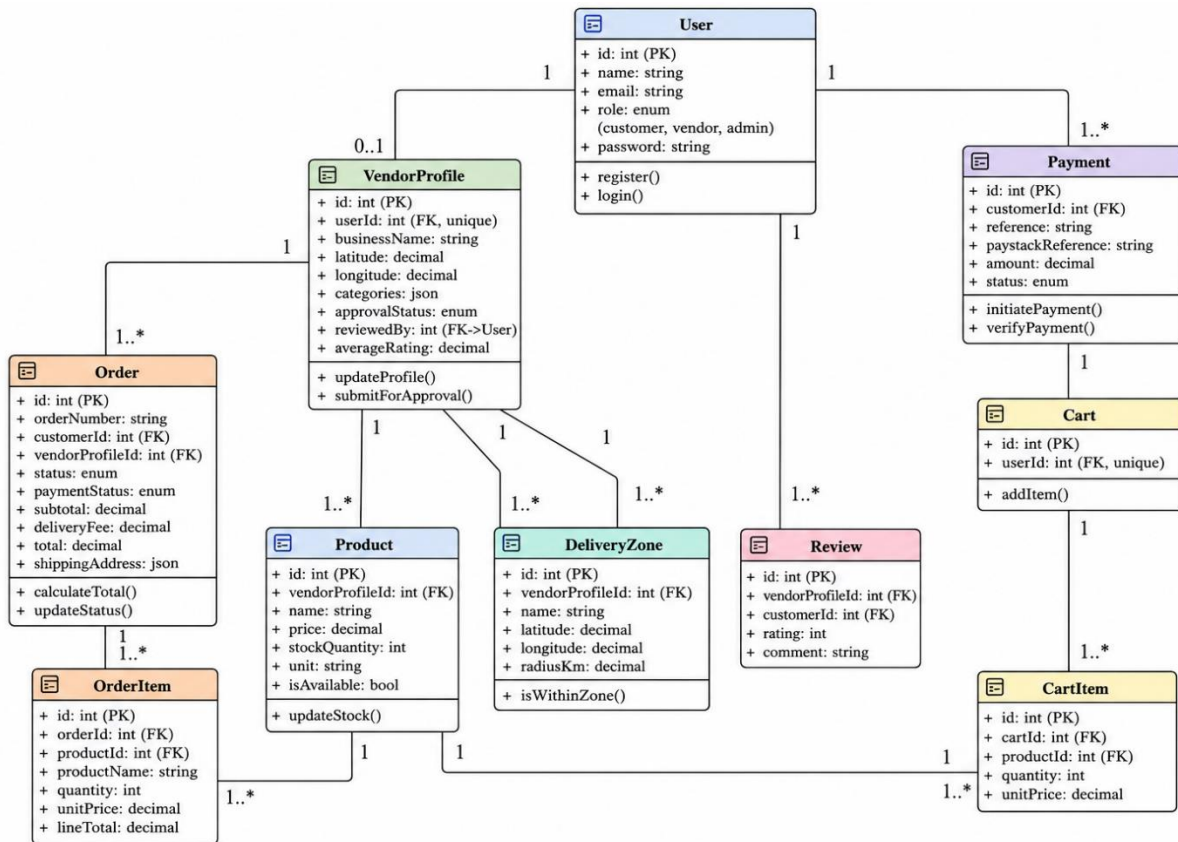


Figure 3.5: Class Diagram of the Sachet and Bottled Water Distribution System

3.4.2 Database Design

The database supporting the system is implemented in MySQL (MariaDB 10.4) and managed through Laravel's migration system, and was designed to satisfy at least the Second Normal Form (2NF). A relation is in 2NF when it is first in First Normal Form (1NF) every attribute holding a single, atomic value, with no repeating groups and every non-key attribute depends on the whole of the relation's primary key, not merely part of it. Because every table in this schema is keyed by a single-column surrogate identifier (an auto-incrementing id) rather than a composite key, no non-key attribute can ever depend on only part of the primary key, so every relation satisfies 2NF by construction; the design task was therefore to keep each table's non-key columns describing only that one entity, with any data shared across entities pulled out into its own table and referenced by a foreign key rather than being copied in.

Two deliberate exceptions are worth noting because they can look, at first glance, like redundancy. First, `order_items` store its own `product_name` alongside `product_id`, even though

the name is already recorded in products; this is an intentional snapshot so that a past receipt still shows the name a product had at the time of purchase even if the vendor later renames or removes it a common and accepted pattern for order history tables. Second, orders stores a total that is arithmetically derivable from subtotal and delivery fee; storing it avoids recomputing the figure on every read and preserves the exact amount that was actually charged. Neither exception violates 2NF, since both columns still depend on their own table's whole primary key, they are deliberate historical/performance choices, not accidental partial dependencies. A small number of columns vendor profiles. categories, products. Attributes, orders. shipping address, and payments. Metadata store JSON-formatted, semi-structured data (for example, a vendor's list of product categories, or a delivery address broken into line1/city fields together with geocoordinates). These are treated as single, opaque application-level values rather than decomposed into further relations, which keeps the schema flexible for vendors with different attribute needs without requiring a separate table for every possible attribute.

Figure 3.6 shows the Entity-Relationship Diagram for the core, domain-relevant tables of the database (framework-level Laravel tables such as cache, jobs, sessions, and personal_ access tokens, which support authentication and background processing rather than the water-ordering domain itself, are omitted from the diagram for clarity), after which the relational

schema is presented showing each table's attributes and keys exactly as implemented.

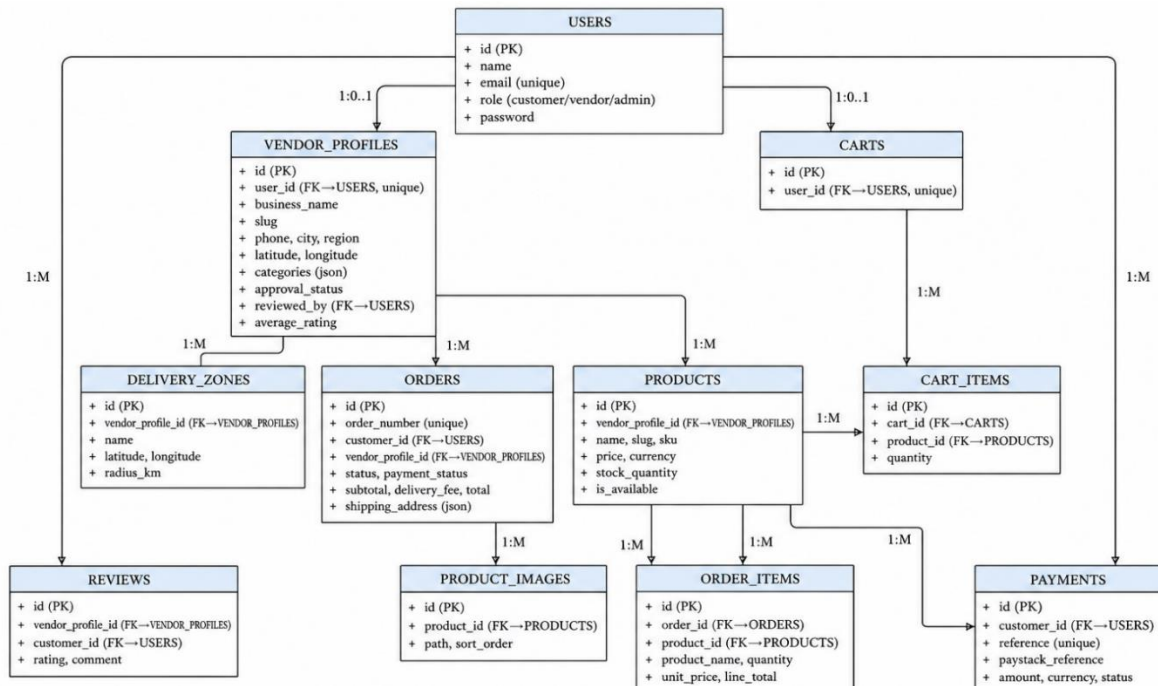


Figure 3.6: Entity-Relationship Diagram (2NF) of the marketplace Database

In this schema, a single user's table carries all three system roles distinguished by the role column, rather than three separate role tables; this matches how the application actually authenticates and authorises requests through Laravel Sanctum. Only users with the vendor role gain a matching row in vendor profiles, linked one-to-one through the unique user id foreign key, which carries business details, geocoordinates for the Google Maps dashboard, a JSON list of product categories (which, for this project, is populated with values such as sachet water and bottled water), and an approval status workflow through which an admin (recorded in reviewed by) approves or rejects a new vendor. A vendor profiles row relates to many delivery zones (each a centre point and radius used to check whether a customer's delivery location is serviceable) and many products, each tagged as sachet or bottle water through its name and the JSON attributes column. A customer's in-progress selections are held in a one-to-one cart per user, exploded into cart items rows once a product is added, while a placed order is recorded in orders and its individual line items in order items resolving the many-to-many relationship between orders and products. Each order can accumulate multiple payments rows (to capture retried or failed attempts, as seen in the sample data) identified by Paystack's

own reference and access code, and a customer may leave at most one review per vendor, enforced by the unique constraint on (customer id, vendor profile id).

CHAPTER FOUR

SYSTEM DEVELOPMENT AND TESTING

4.1 Implementing of the Design

The implementation phase translated the design decisions from Chapter Three into working code for the Sachet and Bottled Water Distribution System. Development followed an iterative and incremental model, in which each module registration and login, vendor product management, location-based vendor discovery, cart and checkout, payment processing, and the admin dashboard was built, tested on its own, and then integrated into the wider system. Building the system in small, verifiable increments made it possible to catch integration problems for example, a mismatch between a Flutter request and a Laravel route early, rather than discovering them only after the whole application was assembled.

The customer and vendor mobile applications were implemented in Flutter and Dart, using Visual Studio Code and Android Studio as the primary development environments, with Android emulators and iOS simulators used to verify behavior on both target platforms. The PHP backend was implemented on the Laravel framework, with Eloquent used to define the models corresponding to the users, vendor profiles, delivery zones, products, carts, cart items, orders, order items, payments, and reviews tables designed in Chapter Three, and Laravel's migration system used to create and version the MySQL schema. Laravel Sanctum was configured to issue and validate the personal access tokens that authenticate requests from the mobile app, so that only a logged-in customer, vendor, or admin can reach the endpoints permitted for their role.

Input validation was applied both on the Flutter side, to give the user immediate feedback (for example, rejecting an empty product price or an invalid phone number before a request is even sent), and on the Laravel side, using Laravel's built-in request validation rules, so that the backend never trusts data arriving from the client without checking it first. Error handling on the backend made consistent use of structured try-catch style exception handling around database operations and external API calls, returning a predictable JSON error response rather than allowing the application to crash. On the mobile side, network and payment failures are

caught and surfaced to the user as a clear message with the option to retry, rather than leaving the app in an unclear state.

Integrating the three payment channels required particular care. The Paystack Application Programming Interface was connected using its official PHP SDK on the backend, with the mobile app opening Paystack's hosted checkout (which itself offers card and MTN Mobile Money) for a transaction, direct mobile-money channel, and ATM/debit or credit card payment was supported through Paystack's card-processing rails. In every case, the backend never stores raw card or mobile-money credentials; it stores only the transaction reference, the Paystack access code, and the payment status returned by the gateway, consistent with the security requirement specified in Section 3.1.2. The Google Maps Platform was integrated on the Flutter side to request and display the customer's device location and nearby vendors on a map, and on the web-based admin dashboard to plot vendor, delivery-zone, and order locations as pins that an administrator can inspect.

JSON was used as the standard data-interchange format across the whole system: the Flutter app sends and receives JSON over HTTPS, Laravel encodes and decodes JSON for the semi-structured columns identified in Chapter Three (categories, attributes, shipping_address, and metadata), and Paystack with the backend using JSON payloads, giving the system one consistent format from the mobile client through to the external payment gateways.

4.1.2 Programming and Coding

The programming phase made use of the following languages, frameworks, and libraries:

1. Frontend (Mobile): Flutter and Dart, used to build the Customer and Vendor applications for Android and iOS from one codebase.

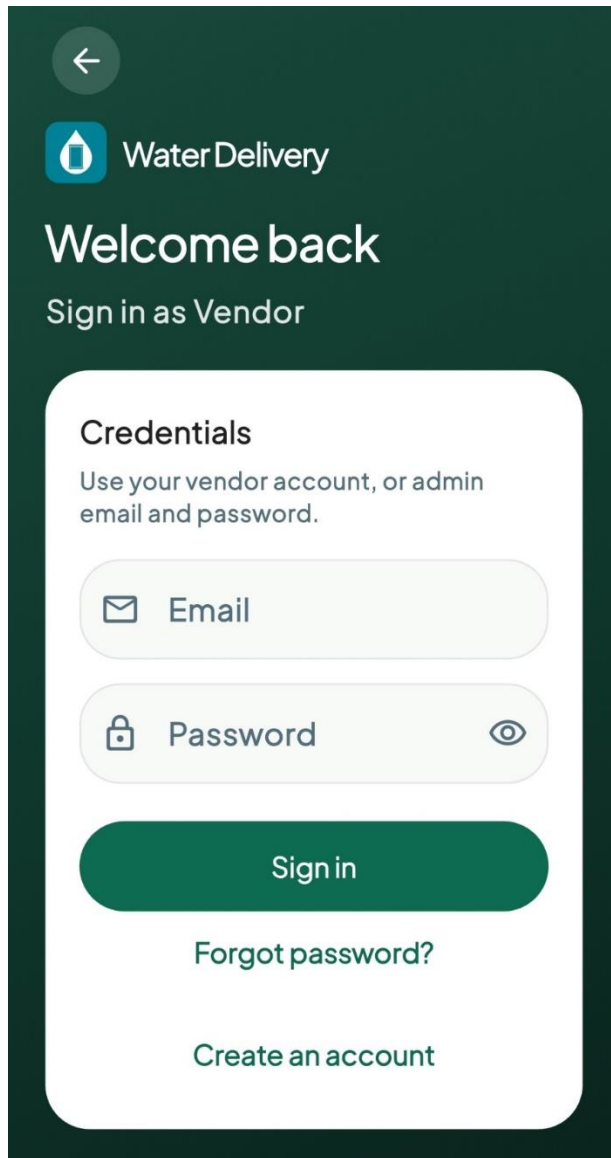
Backend: PHP on the Laravel framework, providing routing, Eloquent ORM, request validation, and Laravel Sanctum for token-based API authentication.

2. Database: MySQL (MariaDB), accessed through Laravel's query builder and Eloquent models, with migrations used to create and update the schema designed in Chapter Three.

3. Payments: The Paystack PHP SDK for card and mobile-money checkout, Cash Application Programming Interface for direct mobile-money payments.
4. Location and Mapping: the Google Maps Platform (Maps SDK for Flutter, Geocoding, and Places Application Programming Interfaces) for capturing device location, displaying nearby vendors, and rendering the admin map dashboard.
5. Security: Laravel Sanctum for authentication, bcrypt password hashing, HTTPS/TLS for all client-server traffic, and Laravel's built-in validation and query-parameter binding to guard against injection.

Coding practice throughout the project emphasized consistent input validation, structured error handling, and clear separation between the Flutter presentation layer and the Laravel API layer, with JSON as the sole format for data exchanged between them. No third-party package was used for the core vendor-matching logic beyond Laravel's own query builder and the Google Maps Platform's distance/geocoding utilities. Screenshots of the main program files and representative code snippets including the Flutter screens, Laravel routes, and Eloquent model definitions are inserted below as evidence of implementation.

Figure 4.1 displays the vendor mobile authentication interface. The form provides input fields for business email and password, using client-side validation to check input formatting before sending authentication requests to the Laravel backend via Sanctum.



The image shows a mobile app interface for 'Water Delivery'. At the top left is a back arrow icon. Below it is the app logo, which consists of a blue square with a white water drop icon, followed by the text 'Water Delivery'. The main heading is 'Welcome back' in a large, white, sans-serif font. Below this is the text 'Sign in as Vendor' in a smaller, white, sans-serif font. The central part of the screen is a white rounded rectangle containing the login form. It starts with the title 'Credentials' in bold. Below the title is a subtitle: 'Use your vendor account, or admin email and password.' There are two input fields: the first is labeled 'Email' with an envelope icon on the left; the second is labeled 'Password' with a lock icon on the left and an eye icon on the right to toggle visibility. Below these fields is a large, dark green rounded button with the text 'Sign in' in white. At the bottom of the white box are two links: 'Forgot password?' and 'Create an account', both in a dark green color.

←

 Water Delivery

Welcome back

Sign in as Vendor

Credentials

Use your vendor account, or admin email and password.

 Email

 Password 

Sign in

[Forgot password?](#)

[Create an account](#)

Figure 4.1: Vendor Registration and Login

Figure 4.2 shows the Flutter implementation for the VendorProductDetailScreen widget. The code extracts product metadata (such as name, stock count, description, and currency), applies role-based theming, and renders the layout on the vendor interface.

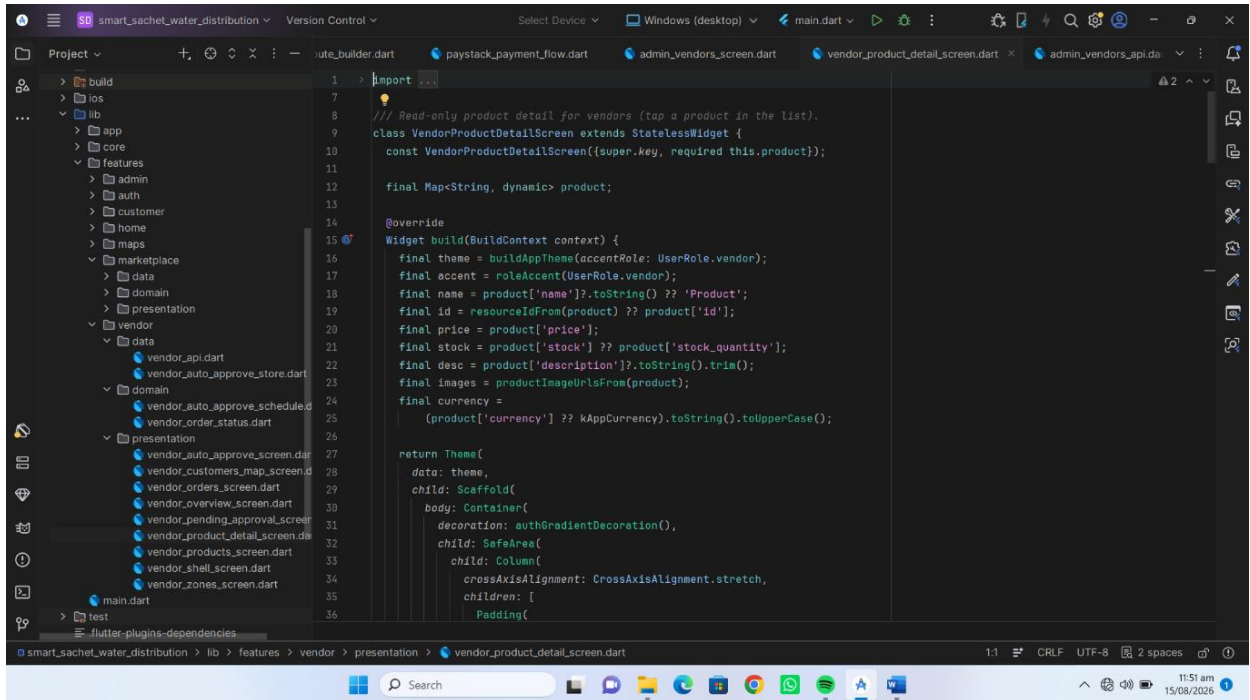


Figure 4.2: Vendor Product

Figure 4.3 shows the Dart implementation for handling order checkouts. The functions parse unique order reference codes and prepare payment payloads before launching the Paystack payment gateway.

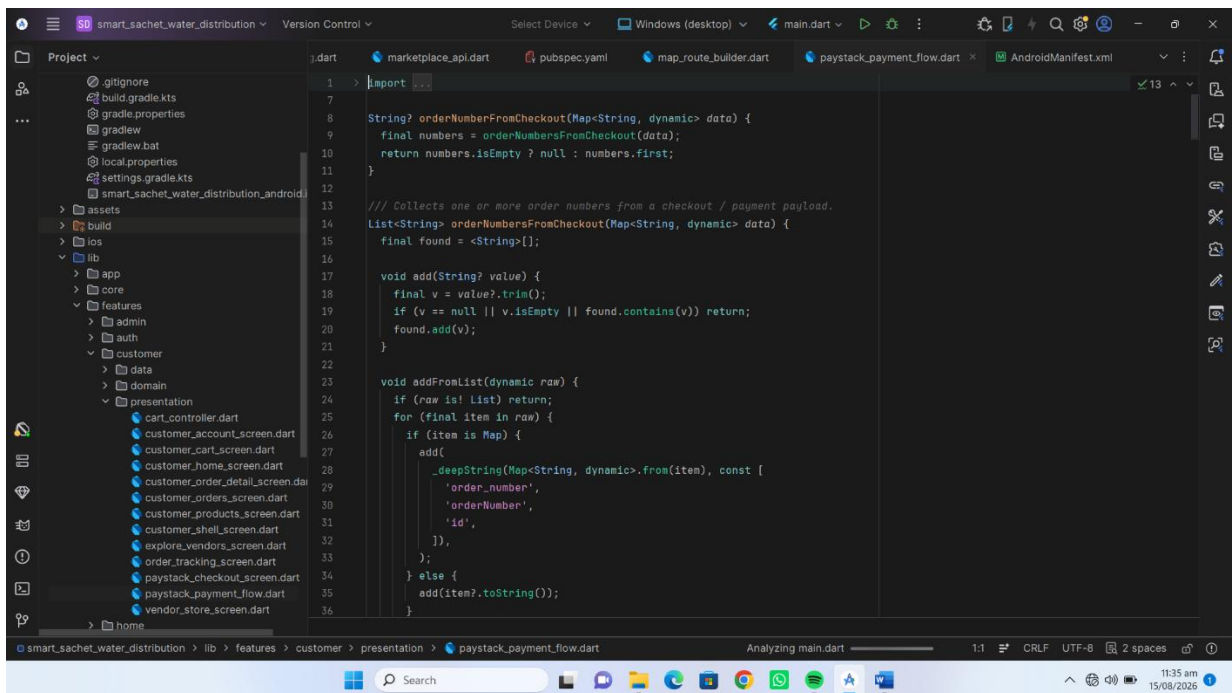


Figure 4.3: Order Placement and Paystack Payment Initiation Route

Figure 4.4 shows the Flutter implementation of the AdminVendorsScreen stateful widget. The code initializes vendor data arrays, handles asynchronous API queries, and filters accounts by their approval status (pending, approved, or rejected).

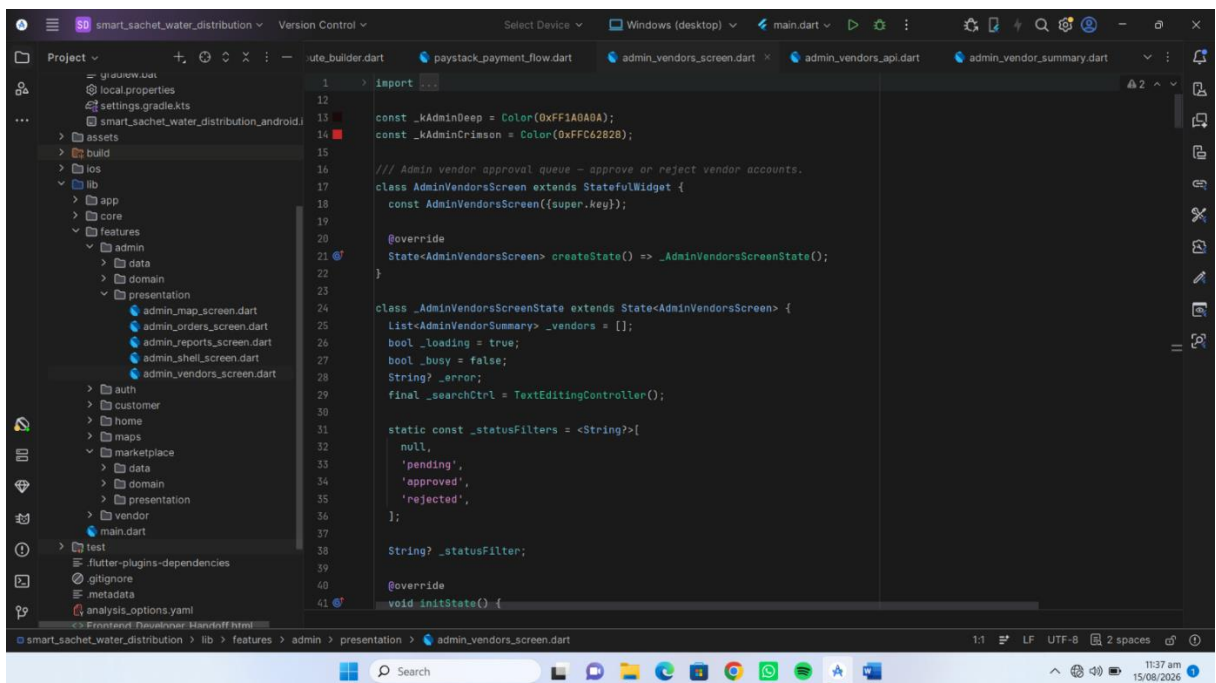


Figure 4.4: Admin Vendor-Approval Page

Figure 4.5 shows the database table structure in phpMyAdmin. It shows the orders table columns, foreign key relations, monetary decimal types, and an active order linked to customer and vendor accounts.

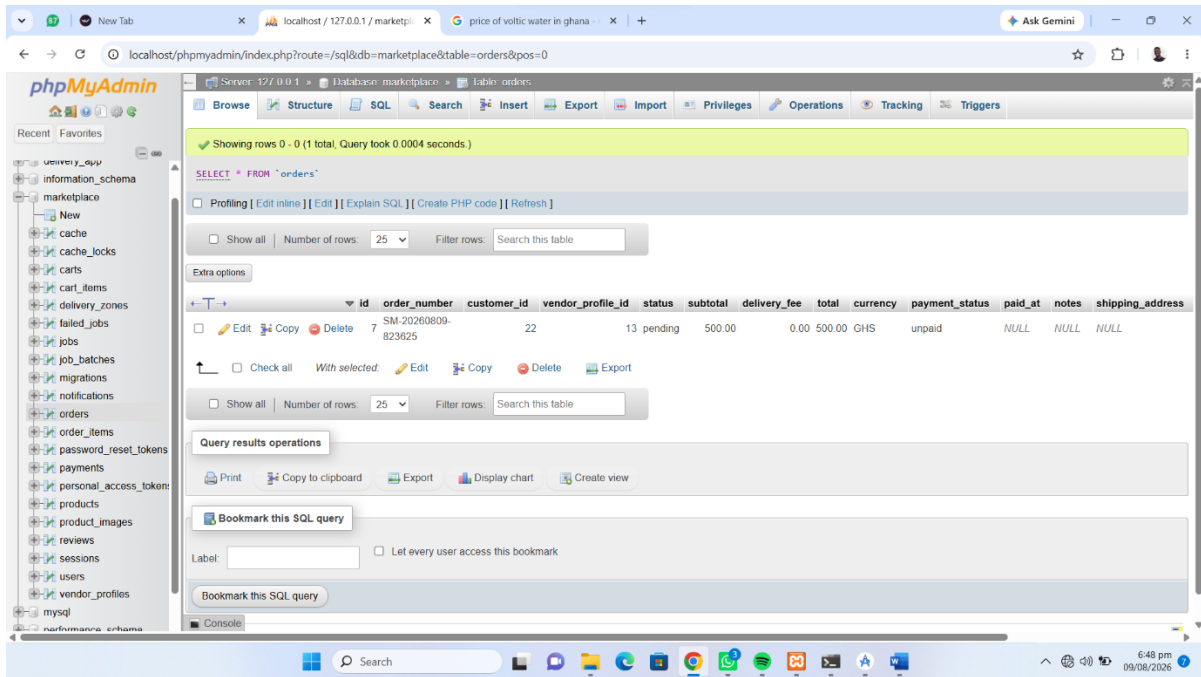


Figure 4.5: Eloquent Model / Database Migration for the Orders Table

4.1.3 Main User Interfaces

The system's interfaces were designed around the three user roles established in Chapter Three which are the Customer, Vendor, and Administrator with each role seeing only the screens and actions relevant to it.

The Customer interface allows a user to register or log in, view nearby sachet and bottle water vendors on a map or list generated from the device's location, browse a vendor's products, add items to a cart, check out and pay through Paystack and track an order from confirmation through to delivery. The Vendor interface allows a registered vendor to log in, manage a catalogue of sachet and bottle water products, define delivery zones, view and update incoming orders, and see basic sales figures and customer ratings. The Administrator interface, delivered as a web-based dashboard embedding the Google Maps Platform, allows an admin to review

and approve or reject new vendor applications, monitor vendor, delivery-zone, and order locations on a map, and manage user accounts.

Across all three interfaces, forms include inline validation and clear error feedback (for example, flagging an invalid phone number or a product price of zero before the request reaches the server), and file or image uploads (such as a vendor's product photo) are restricted to accepted formats and a maximum size before being accepted.

Figure 4.6 shows the customer home screen, displaying cart totals, active order summaries, and nearby water vendors matched using the device's location coordinates.

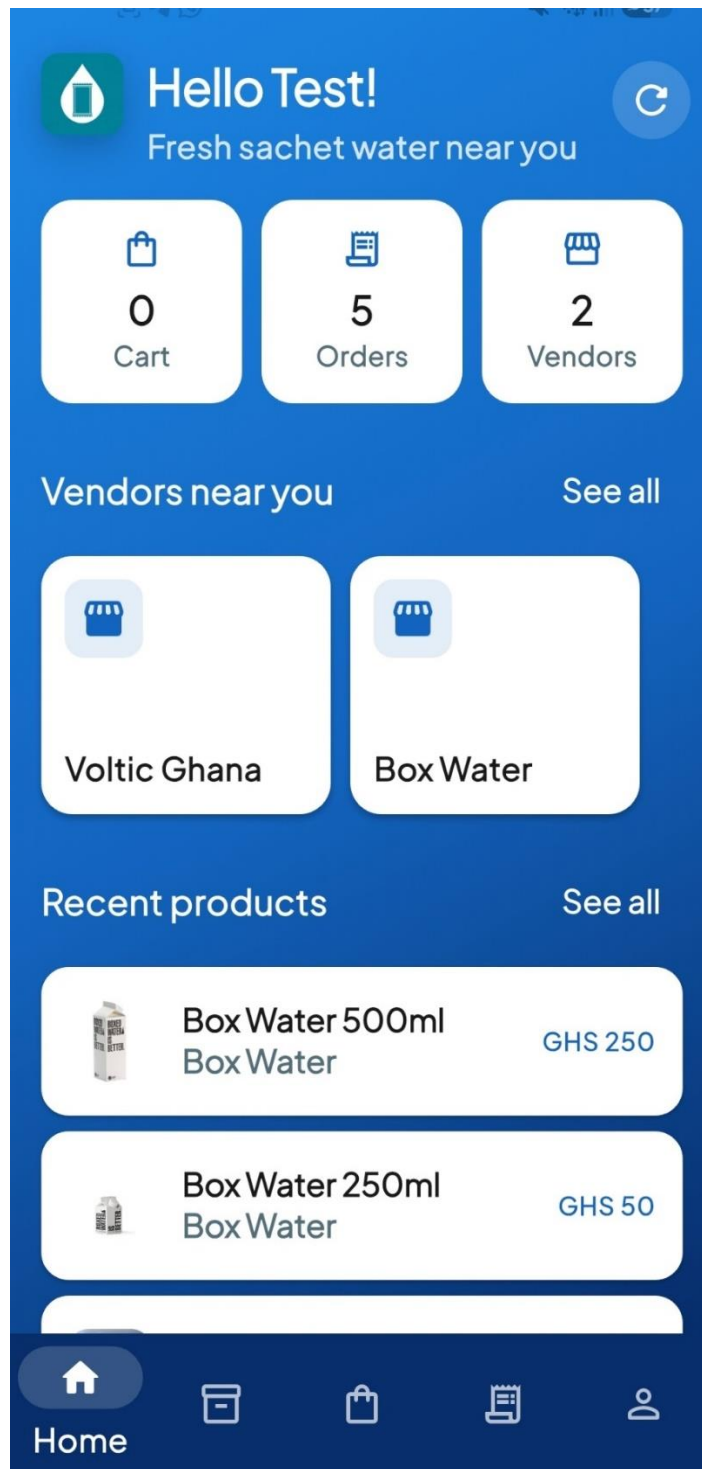


Figure 4.6: Customer App Nearby Vendors / Home Screen

Figure 4.7 shows the customer cart interface, displaying selected water brands, item quantities, sub-total calculations, and buttons to update items or proceed to checkout.

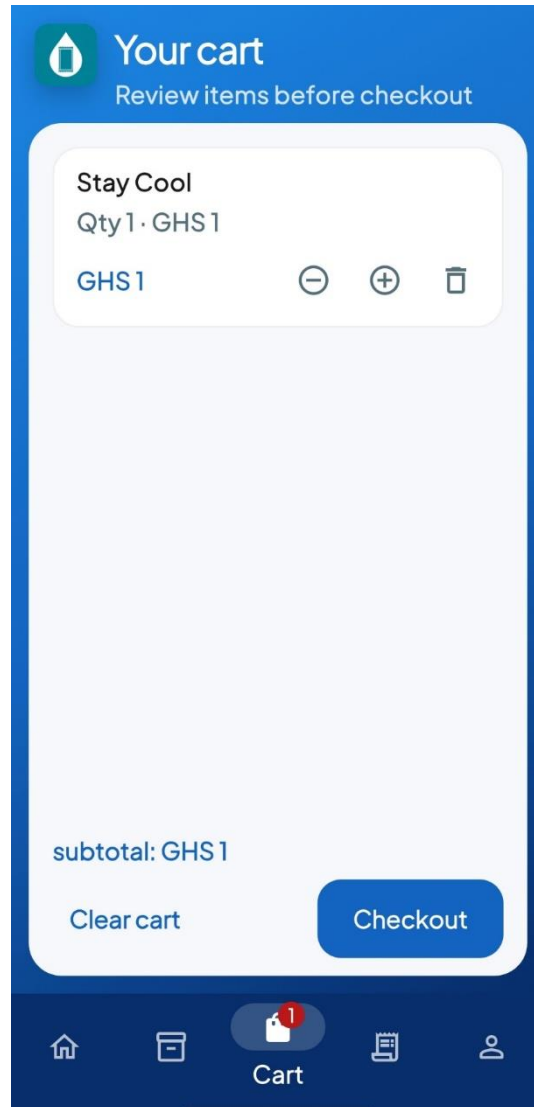


Figure 4.7: Customer App Product Listing and Cart

Figure 4.8 shows the Paystack payment modal within the mobile app, offering multiple payment options including Mobile Money (MTN, Telecel, AT), debit/credit cards, and direct bank transfers.

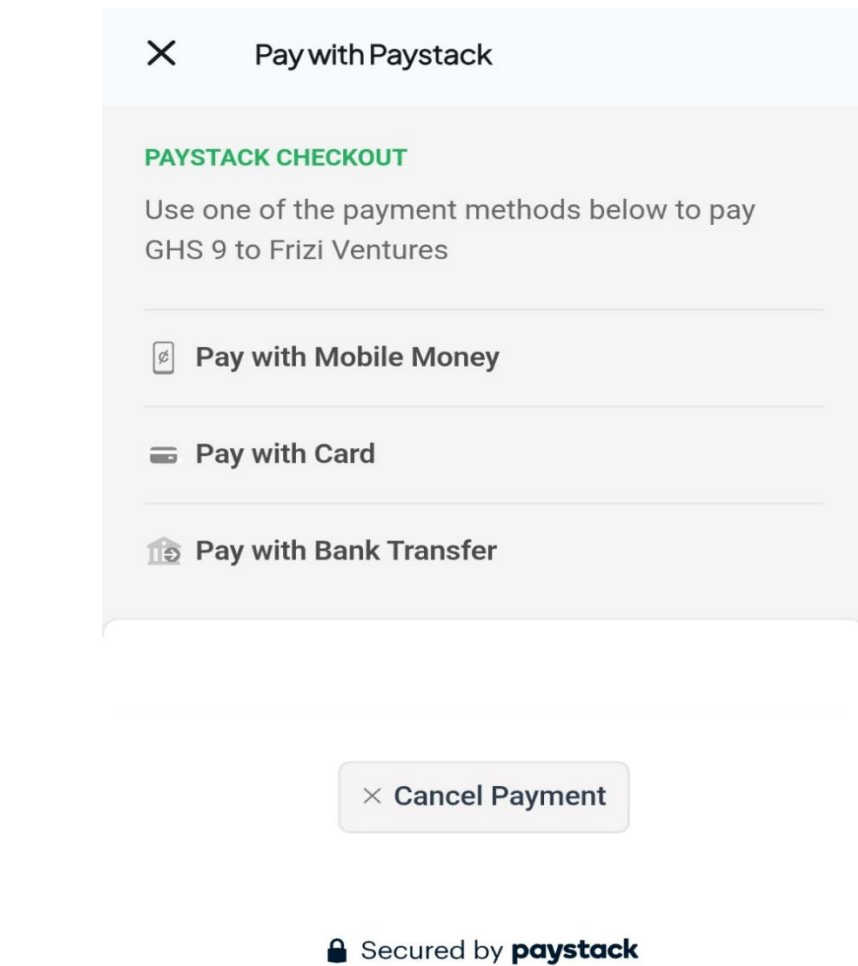


Figure 4.8: Customer App Checkout and Payment Method Selection

Figure 4.9 shows the vendor payout interface, displaying available balances, pending transfers, and an option to request fund withdrawals to a bank account or registered mobile money wallet.

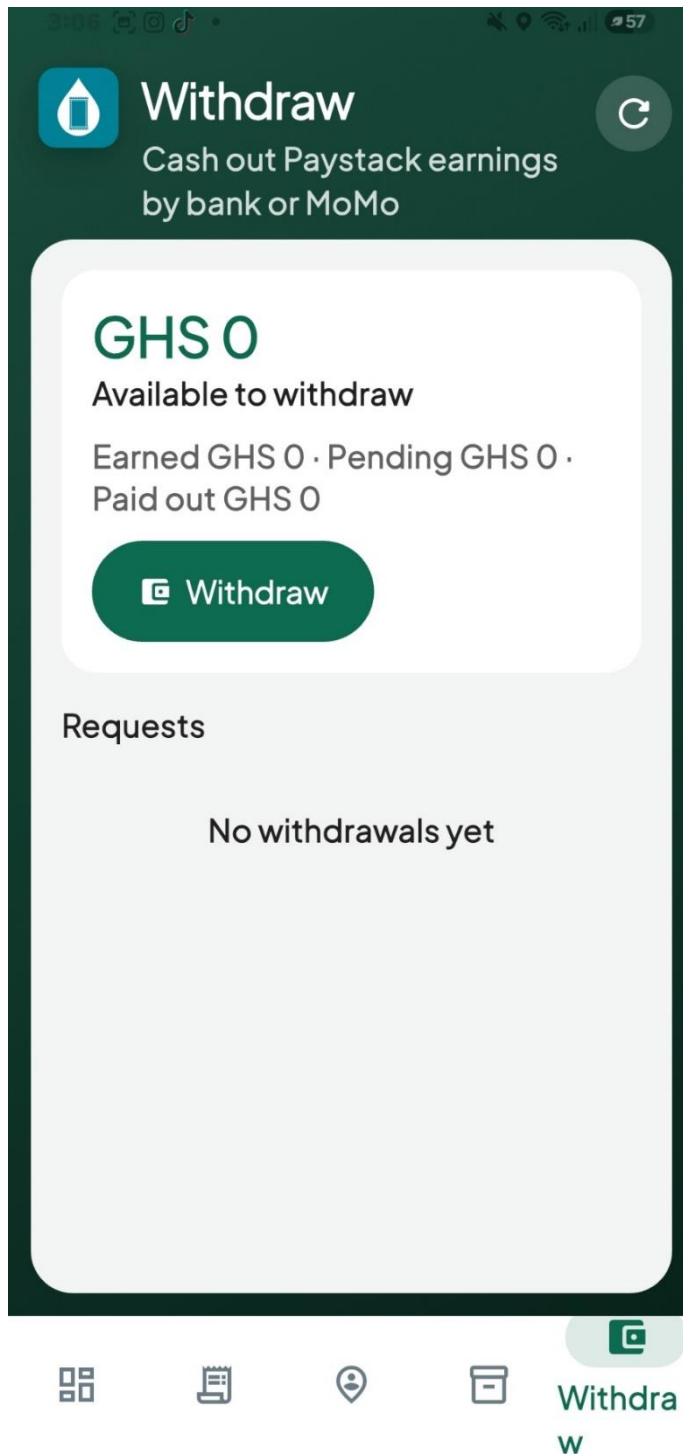


Figure 4.9: Vendor App Money Withdrawal Screen

Figure 4.10 shows the administrative operations map powered by Google Maps. The interface displays geographic markers for active vendor locations and customer orders across Ho Municipality.

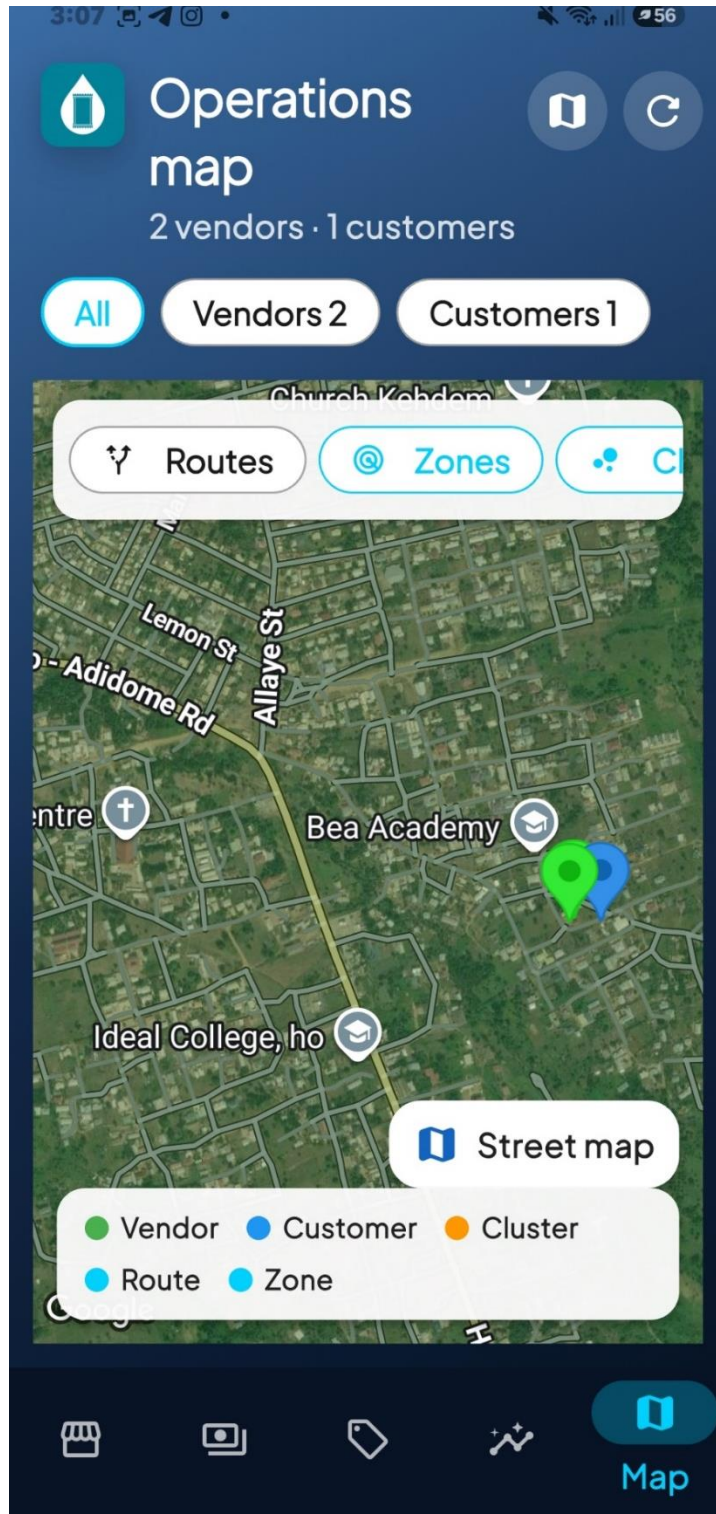


Figure 4.10: Admin Web Dashboard Google Maps Vendor/customer Monitoring

Figure 4.11 shows the vendor delivery map, showing geographic delivery zones, customer drop-off points, and street routes across Ho Municipality.

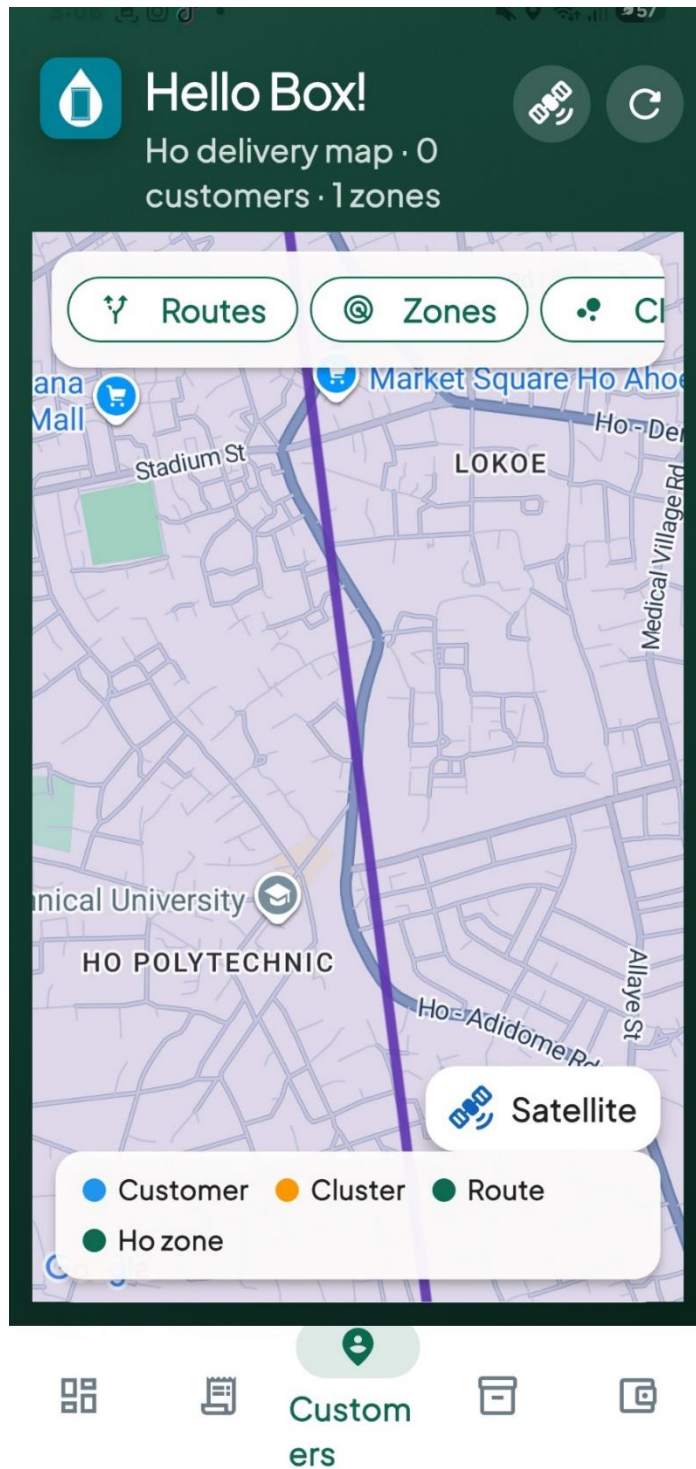


Figure 4.11: Vendor Web Dashboard Google Maps Order Monitoring

Figure 4.12 shows the application landing screen, allowing users to enter either the Customer ordering portal or the Vendor management portal.

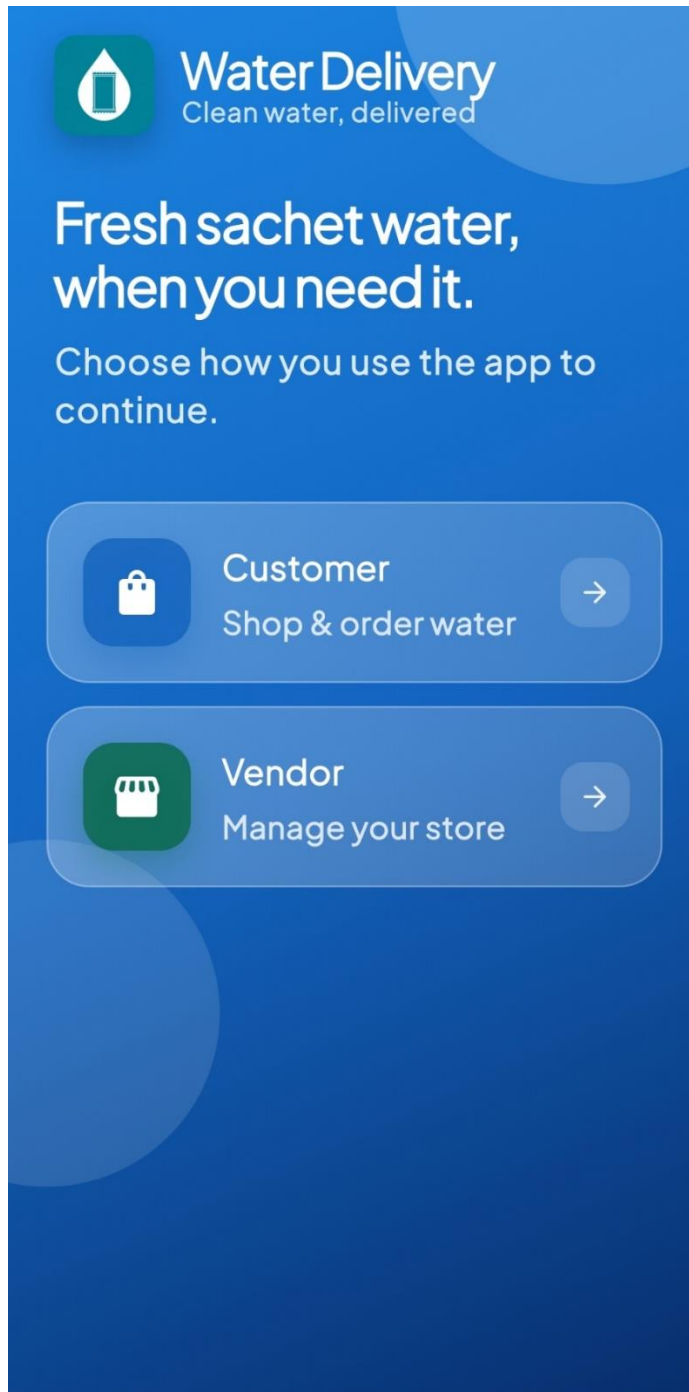


Figure 4.12: Homepage of Application

Figure 4.13 shows the vendor management dashboard, displaying key operational metrics including pending order approvals, active delivery pipelines, live product counts, and customer delivery maps.

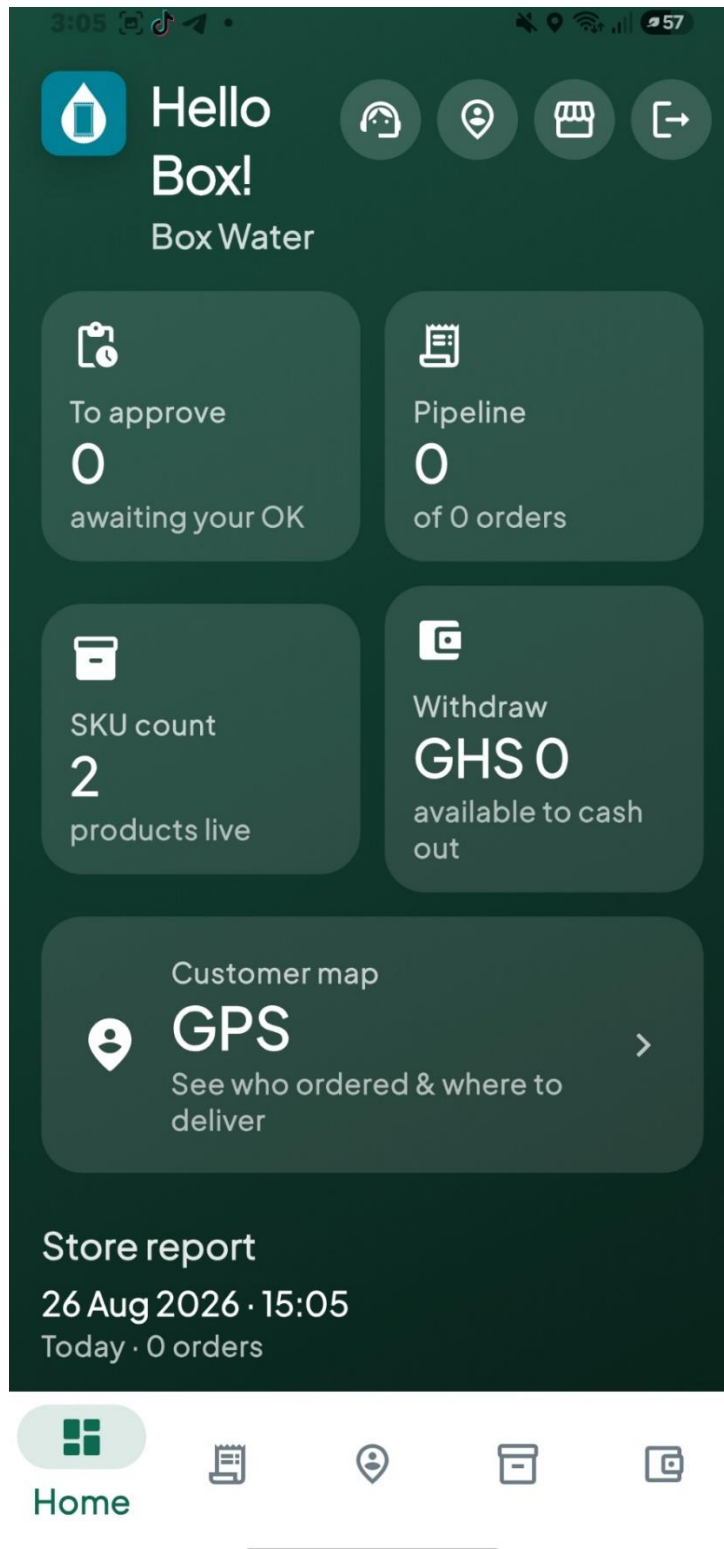


Figure 4.13: Home Page of Vendor Account

Figure 4.14 shows the transaction verification screen on the admin portal, displaying Paystack payment references, payment timestamps, and transaction statuses for completed orders.

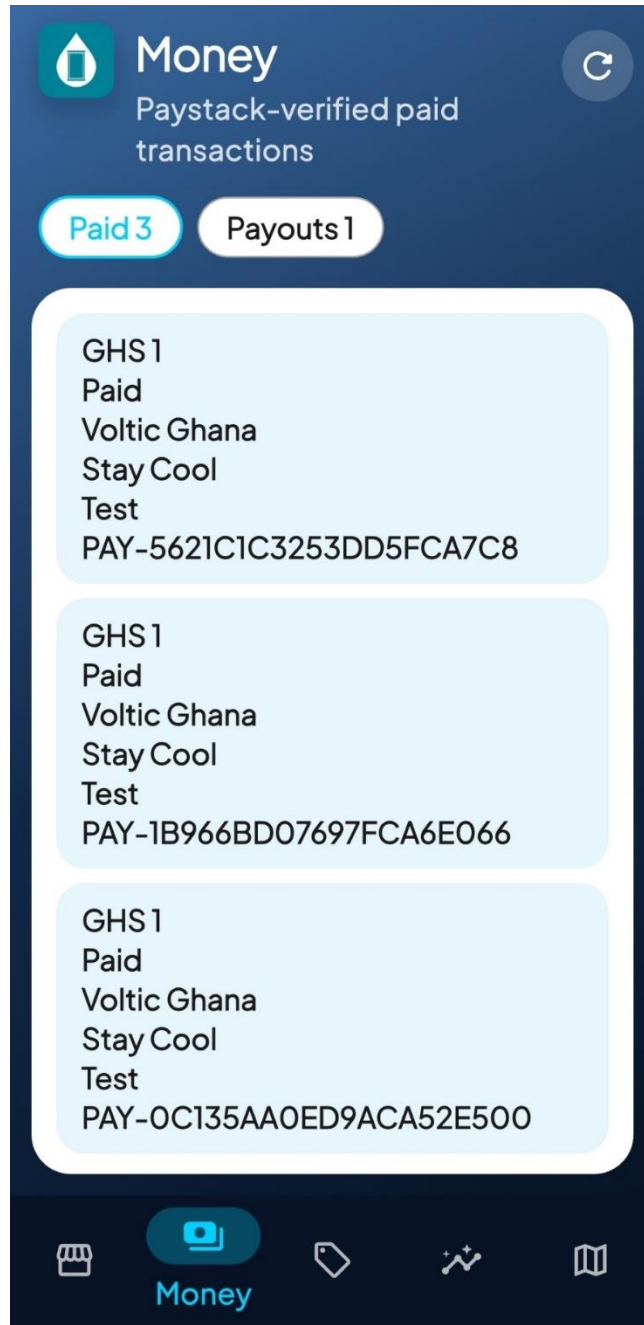


Figure 4.14: Admin Paystack Successful Payment Dashboard

4.2 Testing of the New System

The testing strategy relied primarily on system testing, in which each module was exercised in the context of the running application immediately after it was built, so that problems were caught and corrected before the next module was layered on top. This iterative approach was supplemented, where practical, by manual endpoint testing of the Laravel API using Postman before the corresponding Flutter screen was connected to it, so that backend and frontend issues could be isolated from one another.

Errors encountered during development fell into a small number of recurring categories:

1. Frontend issues in Flutter, such as widgets rebuilding with stale state after an API response, incorrect handling of asynchronous location permission requests, and layout overflow on smaller Android screen sizes.
2. Backend issues in Laravel, including validation rules that were initially too strict or too permissive, Eloquent relationship definitions that returned unexpected null values, and mismatched JSON keys between the Flutter request body and the Laravel controller.
3. Payment integration issues, particularly around correctly handling Paystack's asynchronous webhook callback and reconciling a payment's status with the corresponding order when a transaction was retried after an earlier failure, a scenario also visible in the payments table itself, where more than one payment attempt can be linked to the same order.
4. Location and mapping issues, such as an inaccurate first location reading immediately after the app requested permission, and delivery-zone radius checks that needed adjustment to correctly classify a customer's coordinates as inside or outside a vendor's coverage area.

These issues were resolved through systematic debugging reproducing the error, inspecting Laravel's log output and Flutter's debug console, and correcting the underlying validation, state-management, or query logic followed by re-testing the affected feature before moving on. Table 4.1 summarizes a representative sample of the test cases exercised against the system.

Table 4.1: Sample System Test Cases

ID	Feature Tested	Expected Result	Actual Result	Status
T1	Customer registration	Account is created and a confirmation is returned	Account created successfully; validation blocked duplicate emails	Pass
T2	Nearby vendor discovery	Vendors within range of the device location are listed	Nearby, approved vendors returned and plotted correctly	Pass
T3	Add product to cart	Cart item is created or its quantity updated	Cart and cart_items updated as expected	Pass
T4	Order placement	Order and order_items are created with correct totals	Order created; total matched subtotal + delivery fee	Pass
T5	Paystack payment	Payment reference is created and status updates on callback	Initial attempts failed in sandbox; retry succeeded	Pass (after retry)
T7	Vendor approval workflow	Admin can approve or reject a pending vendor	approval_status updated; vendor notified	Pass
T8	Delivery zone check	Order is accepted only if delivery point is within radius_km	Out-of-zone address correctly rejected	Pass
T9	Admin map dashboard	Vendor and order locations render as pins on Google Maps	Pins rendered; minor delay on first load	Pass
T10	Vendor rating	Customer can submit one review per vendor per order	Duplicate review correctly blocked by unique constraint	Pass

Formal unit and integration test suites were not written; instead, correctness was verified through this repeated manual system testing, with particular attention paid to ensuring that an

order's stored total was always correct, that a delivery address outside a vendor's declared zone was rejected rather than silently accepted, and that a failed or retried payment never left an order in an inconsistent state.

4.3 Documentation

Two levels of documentation were prepared for the system:

Internal Documentation (for developers)

1. Inline code comments within the Flutter widgets and Laravel controllers, models, and migrations, explaining non-obvious logic such as the delivery-zone radius check and the Paystack callback handler.
2. A structured README file covering environment setup, installing PHP and Composer dependencies, running Laravel migrations, and configuring the Paystack and Google Maps Platform API keys.
3. Application Programming Interface documentation, produced from the Postman collection used during development, listing each endpoint together with its required inputs, authentication requirements, and expected JSON response.

User Documentation (for customers, vendors, and administrators)

1. A short in-app guide and onboarding screens for customers, covering how to find nearby vendors, place an order, and pay.
2. A vendor user guide explaining how to complete the approval process, list sachet and bottle water products, define delivery zones, and manage incoming orders.
3. An administrator manual describing how to review vendor applications, use the Google Maps dashboard to monitor activity, and manage user accounts.

Because customers, vendors, and administrators are all direct users of the deployed application, all three groups were provided with user-facing documentation, unlike systems where only staff interact with the software directly.

4.4 Limitation of the Project

A number of factors limit the depth and coverage of this project:

1. Time constraint: the project is undertaken within a single academic year alongside other coursework, which restricts the scale of testing and the number of real vendors and customers that can be involved during development.
2. Device and connectivity dependence: the system assumes that users have access to an Android or iOS smartphone with an active internet connection; residents who rely solely on basic feature phones or who have limited or unstable internet access may not be able to fully benefit from the platform.
3. Location accuracy: because the system relies on a smartphone's built-in location services rather than dedicated GPS receiver hardware, the estimated location of a customer or vendor may occasionally be less precise than a professional-grade navigation system, particularly in areas with weak network coverage.
4. Vendor participation: the usefulness of the multi-vendor model depends on the willingness of sachet and bottle water companies in Ho Municipality to register and actively maintain their listings; the project cannot compel adoption by existing producers.
5. Payment gateway constraints: development and testing of the Paystack integration are largely carried out in a sandbox or test environment; full production deployment would require each participating vendor to complete Paystack's business verification and settlement account setup.
6. Resource and infrastructure limitations: as a student project, the system is tested on a limited number of devices and a small simulated user base rather than under full commercial load, so performance under large-scale, real-world traffic is not exhaustively evaluated.
7. Logistics realism: actual delivery riders are not commercially engaged during the project; delivery coordination features are demonstrated and tested using simulated or volunteer scenarios rather than a live fleet of riders.

Taken together, these limitations mean that the system produced at the end of this project should be understood as a working prototype and proof of concept for a Sachet and Bottle Water Distribution System, rather than as a finished, commercially deployed product. Nevertheless, the prototype is designed to demonstrate, in a practical and verifiable way, that the core objectives of multi-vendor onboarding, mobile-location-based vendor discovery, order coordination, and secure Paystack-based payment can be achieved using the chosen technology stack of Flutter, PHP, and MySQL within the context of Ho Municipality.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This project set out to design and implement a Sachet and Bottled Water Distribution System that would replace the fragmented, single-vendor, manual ordering practices identified in Ho Municipality with a single, multi-vendor mobile platform. That objective has been met. The system brings customers, registered sachet and bottle water vendors, and a platform administrator together on one shared application, built with Flutter so that a single codebase serves both Android and iOS.

On the server side, the system was implemented on a PHP/Laravel backend with a MySQL database designed and verified to satisfy the Second Normal Form, giving the platform a schema covering users, vendor profiles, delivery zones, products, carts, orders, and payments that is both consistent with sound relational design practice and grounded in a working database already in use for the application. Customers can discover nearby vendors through their device's own location services rather than dedicated GPS hardware, add sachet or bottle water products to a cart, and pay through any of three channels which is Paystack while vendors manage their product catalogue, delivery zones, and incoming orders, and an administrator approves new vendors and monitors activity across Ho Municipality through a web dashboard built on the Google Maps Platform.

Development followed an iterative and incremental approach, in which each module was built and exercised as soon as it was implemented, allowing integration problems between the Flutter client and the Laravel API to be caught and corrected early rather than accumulating until a single, harder-to-diagnose testing phase. System-level testing, summarised in Chapter Four, confirmed that the platform's core flows registration, vendor discovery, cart and checkout, multi-channel payment, the vendor-approval workflow, and delivery-zone enforcement behave as designed. Formal automated unit and integration test suites and quantitative performance benchmarking were not carried out within the scope of this project, and are identified below as the most valuable next step for anyone extending this work.

Taken as a whole, the project demonstrates that a modest, well-scoped combination of proven, widely documented technologies like Flutter, Laravel, MySQL, Paystack, and the Google Maps Platform is sufficient to replace the informal, single-vendor ordering channels documented in Chapter Two with a functioning, multi-vendor alternative, and it provides a working foundation on which the recommendations in the next section can be built.

5.2 Recommendations

To strengthen the system beyond the scope of this project, the following recommendations are proposed:

1. **Formal Automated Testing:** introduce unit and integration test suites (for example, PHPUnit for the Laravel backend and Flutter's own widget/integration testing framework) so that reliability and regressions can be verified automatically rather than through manual system testing alone.
2. **Performance and Load Testing:** measure concrete performance metrics API response time under concurrent orders, time to first location fix, and payment-confirmation latency to give quantitative assurance that the platform can handle peak demand as the number of vendors and customers grows.
3. **Complete the USSD/SMS Channel:** fully implement and pilot the USSD/SMS ordering path scoped in Chapter Three, so that customers using keypad phones can genuinely place and pay for orders, not only smartphone users.
4. **Geographic Expansion:** extend the platform beyond Ho Municipality to other municipalities in the Volta Region and eventually, other parts of Ghana, building on the multi-vendor architecture that already allows new, independent vendors to be onboarded without redesigning the core system.
5. **Enhanced Vendor Analytics:** provide vendors with richer sales and demand reporting (for example, daily order volume and popular products) beyond the basic figures implemented in this project, to help them plan production and stock levels.

6. Stronger Account Security: add two-factor authentication and stronger password policies, particularly for vendor and administrator accounts, given that these roles can approve vendors and manage order and payment data.
7. Push Notifications: extend the current in-app/SMS notifications with real-time push notifications so that customers and vendors are alerted immediately when an order's status changes.
8. Expansion of Google Map to all the regions in Ghana: support the whole Ghana in purchasing water from vendors nationwide and access to multiple users as well.
9. Multilingual Support: add support for widely spoken local languages such as Ewe and Twi alongside English, to make the platform more accessible to a broader cross-section of residents in Ho Municipality.
10. Richer User Documentation: supplement the written user guides produced in this project (Appendix C) with short video tutorials or an in-app interactive walkthrough for first-time customers and vendors.
11. Offline Tolerance: improve the Flutter application's handling of intermittent connectivity, for example by queuing an order locally and submitting it automatically once a network connection is restored.

REFERENCE

- Afridigest. (2025). *How Chowdeck plans to succeed where Jumia Food failed*. Retrieved from <https://afridigest.com/chowdeck-plans-succeed-jumia-food-failed/>
- Ahiabor, W. K., & Donkor, E. S. (2025). Microbial safety of sachet water in Ghana: A systematic review. *Environmental Health Insights*, 19, 1–14.
<https://doi.org/10.1177/11786302241307830>
- Clarity Ventures. (2026). *How to build a multi-vendor marketplace: Complete 2026 guide*. Retrieved from <https://www.clarity-ventures.com/marketplace-ecommerce/how-to-build-a-multi-vendor-marketplace>
- Clearwater Africa. (2026). *Water delivery you can count on*. Retrieved from <https://www.clearwaterafrica.com/>
- Faciotech Blog. (2026). *Mobile money integration for Ghana websites*. Retrieved from <https://blog.faciotech.com/mobile-money-payment-integration-ghana>
- GeoPoll. (2024). *Mobile penetration and internet usage in Ghana*. Retrieved from <https://www.geopoll.com/blog/mobile-penetration-and-internet-usage-in-ghana/>
- GhanaResidential.com. (2025). *Clean water delivery service in Accra for homes & businesses*. Retrieved from <https://ghanaresidential.com/water-delivery-service-in-accra/>
- IRJMETS. (2024). Online food delivery system using Flutter. *International Research Journal of Modernization in Engineering Technology and Science*, 6(3), 1120–1126.
<https://www.irjmets.com>
- Laravel. (2026). *Laravel documentation: Eloquent ORM, migrations, and Sanctum*. Laravel LLC. Retrieved from <https://laravel.com/docs>
- Moulds, S., Chan, A. C. H., Tetteh, J. D., Bixby, H., Owusu, G., Agyei-Mensah, S., Ezzati, M., Buytaert, W., & Templeton, M. R. (2022). Sachet water in Ghana: A spatiotemporal

analysis of the recent upward trend in consumption and its relationship with changing household characteristics, 2010–2017. *PLOS ONE*, 17(5), Article e0265167.

<https://doi.org/10.1371/journal.pone.0265167>

Nursobah, N., Saad, M. I., & Kansil, J. A. J. (2024). Implementation of the Flutter framework for developing an e-commerce application. *TEPIAN*, 5(4), 127–135.

<https://doi.org/10.51967/tepian.v5i4.3110>

Oracle Corporation. (2023). *MySQL 8.0 reference manual*. Oracle Corporation. Retrieved from <https://dev.mysql.com/doc/>

Paystack. (2026). *Why choose Paystack*. Paystack Payments Limited. Retrieved from <https://paystack.com/why-choose-paystack>

PHP Group. (2023). *PHP: Hypertext preprocessor manual*. The PHP Group. Retrieved from <https://www.php.net/manual/en/>

Quartey, R. (2023). *How to get bottled water delivered to your home in Accra*. Retrieved from <https://www.quartey.com/writing/bottled-water-dispenser-delivery-accra>

Sharetribe. (2026). *How to build a multi-vendor marketplace in 2026 (5-step guide)*. Retrieved from <https://www.sharetribe.com/how-to-build/multi-vendor-marketplace/>

Statista. (2025). *Ghana: Online marketplaces for physical goods by visitor numbers*. Statista Research Department. Retrieved from <https://www.statista.com/statistics/1340984/popular-online-marketplaces-for-physical-goods-in-ghana/>

TechCabal. (2025). *Bolt Food welcomes Chowdeck's Ghana expansion but questions its strategy*. Retrieved from <https://techcabal.com/2025/04/10/bolt-food-chowdeck-ghana-expansion/>

TechCrunch. (2021). *Ghana's mPharma partners with Ethiopian conglomerate to enter its eighth market*. Retrieved from <https://techcrunch.com/2021/03/10/ghanas-mpharma-partners-with-ethiopian-conglomerate-to-enter-its-eighth-market/>

Techmoonshot. (2026). *Nigeria's food delivery war is over. The quick commerce war is just beginning*. Retrieved from <https://techmoonshot.com/2026/03/27/nigerias-food-delivery-war-is-over-the-quick-commerce-war-is-just-beginning/>

Techpoint Africa. (2026). *Chowdeck retail logistics: How food delivery is evolving in Africa*. Retrieved from <https://techpoint.africa/guide/chowdeck-retail-logistics/>

Wardrop, N. A., Dzodzomenyo, M., Aryeetey, G., Hill, A. G., Bain, R. E., & Wright, J. (2017). Estimating the expansion of sachet water consumption in urban West Africa: A spatial modeling framework. *Global Health Action*, 10(1), Article 1374528. <https://doi.org/10.1080/16549716.2017.1374528>

Wikipedia. (2025a). *Ghana Water Limited*. Wikimedia Foundation. Retrieved from https://en.wikipedia.org/wiki/Ghana_Water_Limited

Wikipedia. (2025b). *Data Protection Act, 2012 (Act 843)*. Wikimedia Foundation. Retrieved from https://en.wikipedia.org/wiki/Data_Protection_Act,_2012

Yo-Kart. (2026). *How to build online multivendor marketplace: All you need to know*. Retrieved from <https://www.yo-kart.com/blog/how-to-build-an-online-multivendor-marketplace/>

APPENDIX

Appendix A: Project Plan

The project was carried out over the course of one academic semester, following the phased schedule summarised in Table A.1 below.

Table A.1: Project Plan and Timeline

Phase	Activities	Duration	Deliverable
1	Problem identification, background research, and objective setting (Chapter One)	Weeks 1–2	Project proposal / Chapter One
2	Literature review and study of related systems (Chapter Two)	Weeks 3–4	Chapter Two
3	Requirement’s specification, technology selection, architecture, and database design (Chapter Three)	Weeks 5–7	Chapter Three, ERD, and schema
4	Backend development (Laravel API, MySQL migrations, authentication)	Weeks 8–9	Working REST API
5	Mobile app development (Flutter customer and vendor apps)	Weeks 9–11	Customer and vendor app builds
6	Payment gateway and Google Maps integration (Paystack and Maps Platform)	Weeks 11–12	Integrated payment and location features
7	Admin web dashboard development	Week 12	Admin dashboard with map view

8	System testing and debugging (Chapter Four)	Weeks 13–14	Test results and Chapter Four
9	Documentation and report writing (Chapter Five, Bibliography, Appendices)	Week 15	Final project report
10	Final review and submission	Week 16	Submitted project

A.2 Tools and Resources Used

Table A. 2: Tools and Resources Used

Resource	Purpose
Android Studio	Primary development environment
Flutter & Dart	Mobile application development for Android + iOS app
PHP (Laravel)	REST API backend development and maintenance
Google Maps Platform	Location services and administrator dashboard
Paystack Dashboard	Payment configuration, webhook testing and transaction monitoring
MySQL	Relational database management (2nd Normal Form)
Microsoft Word	Project report documentation

Appendix B: User Interfaces (More)

This appendix provides additional screenshots of the application's user interfaces, supplementing those already presented in Section 4.1.3.



Water Delivery
Clean water, delivered

**Fresh sachet water,
when you need it.**

Choose how you use the app to
continue.



Customer
Shop & order water



Vendor
Manage your store



B.1: Splash / Onboarding Screen



 Water Delivery

Create account

Customer

Details

Create an account to order water.

 Full name

 Email

 Phone number

 Password 

 Confirm password 

Create account

B.2: Customer Registration Form



Awaiting admin approval

Thanks for registering. An admin must approve your Ghana Card details before you can access the vendor dashboard.

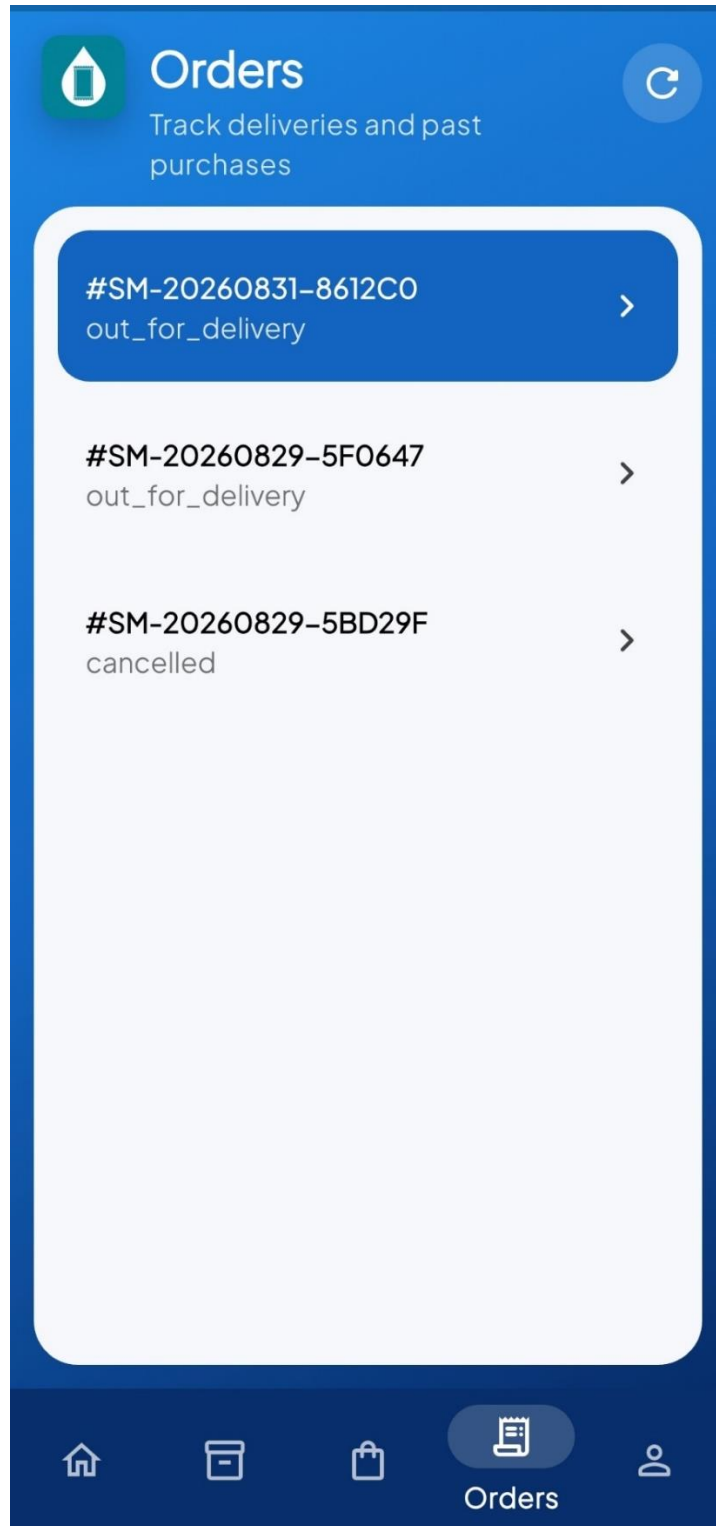
Ghana Card: GHA-274455326-3

 Check approval status

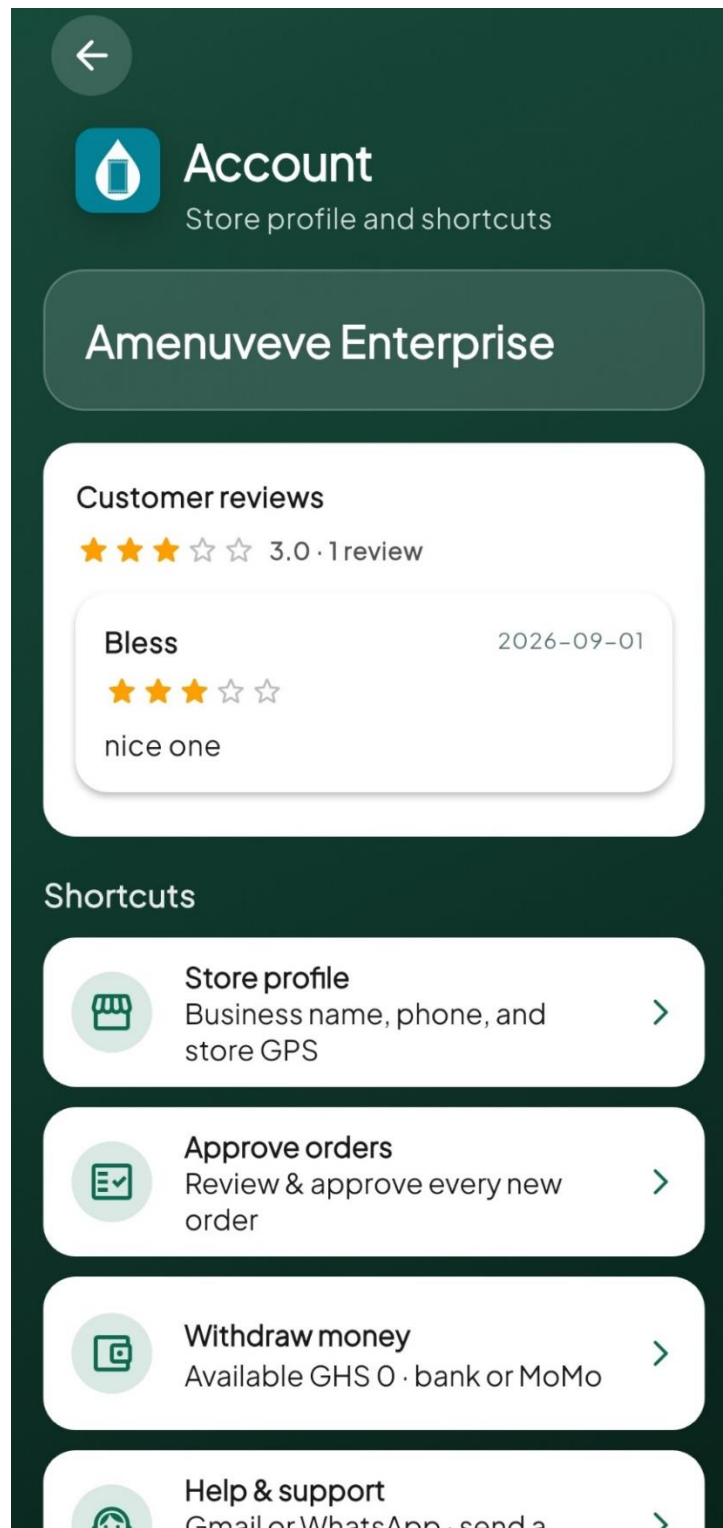
 Help & support

 Sign out

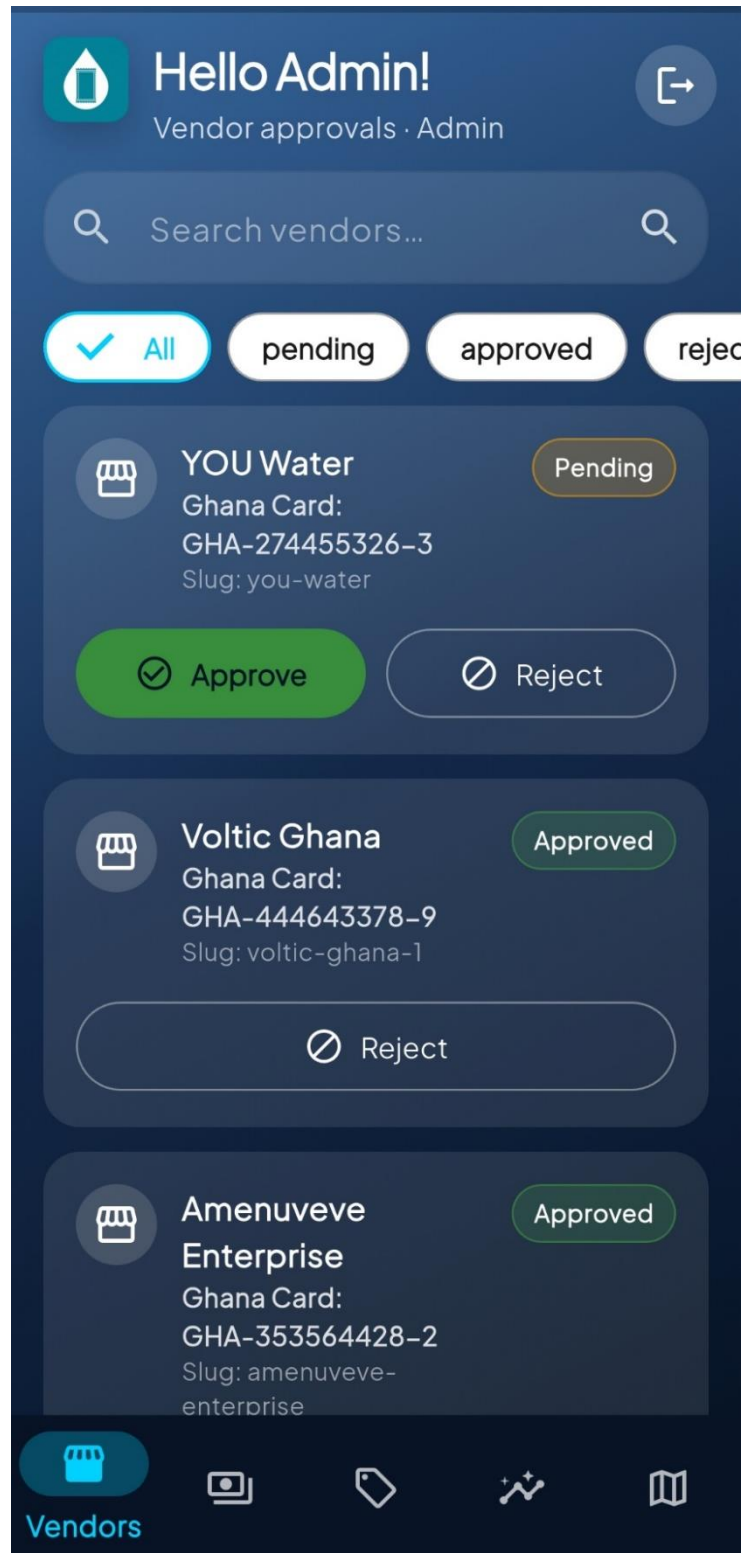
B.3: Vendor Business Profile Submission



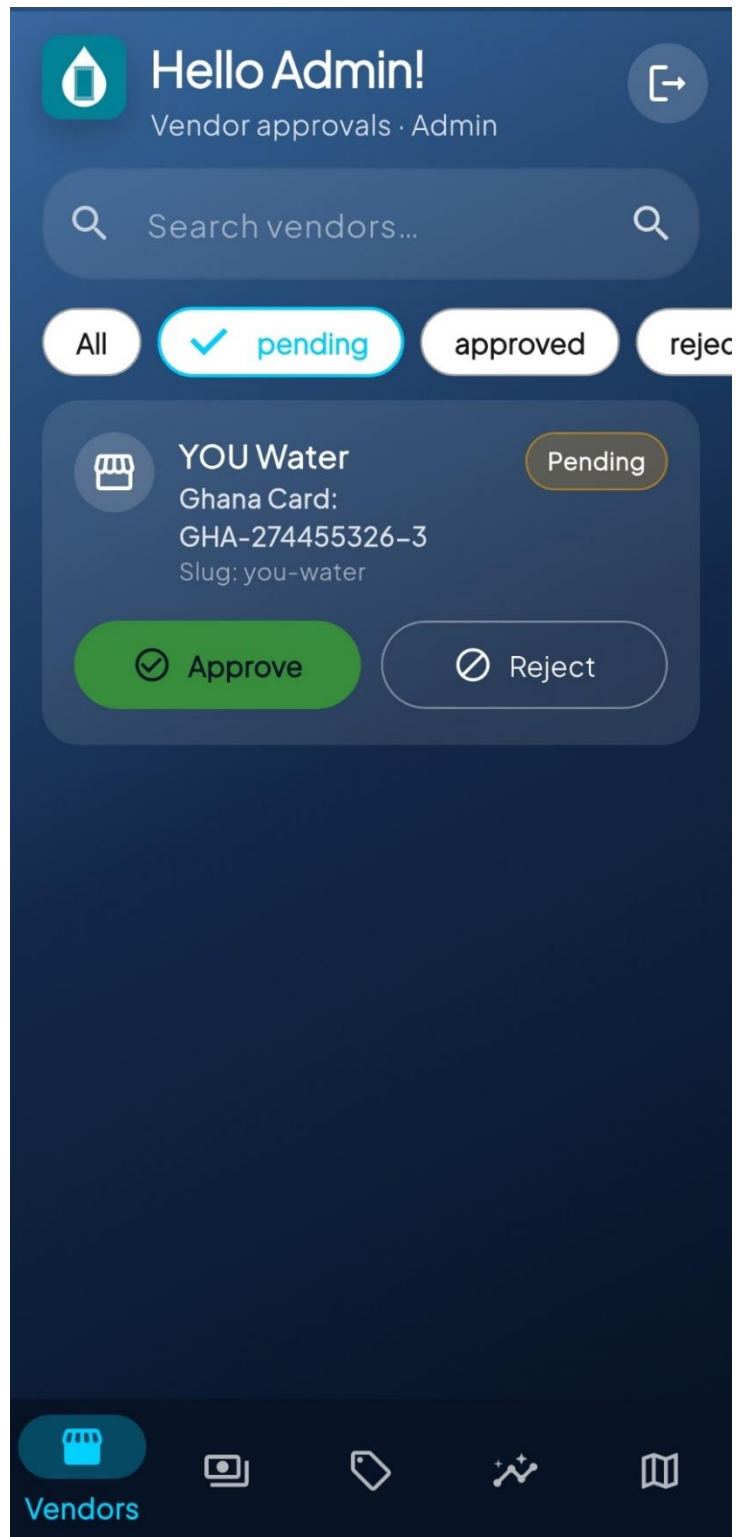
B.4: Order Screen



B.5: Vendor Rating and Review Screen



B.6: Admin User Account Management Screen



B.7: Admin Pending Vendor Approvals List

Appendix C: User Documentation (Guide)

C.1 Customer Quick-Start Guide

1. Download and install the app, then tap “Register” and enter your name, phone number or email, and a password.
2. Allow the app to access your device's location when prompted, so that nearby sachet and bottle water vendors can be shown.
3. Browse the list or map of nearby vendors, open a vendor's page, and add the sachet or bottle water products you want to your cart.
4. Go to your cart, confirm your delivery location, and tap “Checkout.”
5. Choose a payment method which is Paystack (card or mobile money) and complete the payment.
6. Track your order's status from the “My Orders” screen until it is marked as delivered, and rate the vendor afterwards if you wish.

C.2 Vendor Quick-Start Guide

7. Register a vendor account and submit your business details for review by the platform administrator.
8. Once your account is approved, log in and add your sachet and bottle water products, including price and stock quantity.
9. Define one or more delivery zones by setting a centre point and coverage radius on the map, so the app knows which customers you can serve.
10. Monitor the “Orders” screen for new orders, accept them, and update their status as you prepare and dispatch each delivery.
11. Check your sales figures and customer ratings from your vendor dashboard.

C.3 Administrator Quick-Start Guide

12. Log in to the web-based admin dashboard using your administrator credentials.
13. Review pending vendor applications under “Vendor Approvals,” checking the submitted business details before approving or rejecting each one.
14. Use the Google Maps dashboard view to monitor vendor locations, delivery zones, and order activity across Ho Municipality.
15. Manage user accounts: customers, vendors, and fellow administrators from the “Users” section as needed.